Context Constraints for Compositional Reachability Analysis

SHING CHI CHEUNG
Hong Kong University of Science and Technology
and
JEFF KRAMER
Imperial College of Science, Technology and Medicine

Behavior analysis of complex distributed systems has led to the search for enhanced reachability analysis techniques which support modularity and which control the state explosion problem. While modularity has been achieved, state explosion is still a problem. Indeed, this problem may even be exacerbated, as a locally minimized subsystem may contain many states and transitions forbidden by its environment or context. Context constraints, specified as interface processes, are restrictions imposed by the environment on subsystem behavior. Recent research has suggested that the state explosion problem can be effectively controlled if context constraints are incorporated in compositional reachability analysis (CRA). Although theoretically very promising, the approach has rarely been used in practice because it generally requires a more complex computational model and does not contain a mechanism to derive context constraints automatically. This article presents a technique to automate the approach while using a similar computational model to that of CRA. Context constraints are derived automatically, based on a set of sufficient conditions for these constraints to be transparently included when building reachability graphs. As a result, the global reachability graph generated using the derived constraints is shown to be observationally equivalent to that generated by CRA without the inclusion of context constraints. Constraints can also be specified explicitly by users, based on their application knowledge. Erroneous constraints which contravene transparency can be identified together with an indication of the error sources. User-specified constraints can be combined with those generated automatically. The technique is illustrated using a clients/server system and other examples.

Categories and Subject Descriptors: D.2.1 [**Software Engineering**]: Requirements/Specifications; D.2.2 [**Software Engineering**]: Tools and Techniques; D.3.2 [**Programming Languages**]: Language Classifications—*concurrent*, *distributed*, *and parallel languages*; D.3.3 [**Programming Languages**]: Language Constructs and Features—*concurrent programming*

This article is an extended and revised version of previous work [Cheung and Kramer 1994a; 1995a].
This research was partially sponsored by the RGC (HKUST 662/95E), the Swiss Bank, the DTI (IED 410/36/2), and the EPSRC (GR/J 87022).
Authors' addresses: S. C. Cheung, Department of Computer Science, Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong; email: scc@cs.ust.hk; J. Kramer, Department of Computing, Imperial College of Science, Technology and Medicine, London SW7 2BZ, UK; email: jk@doc.ic.ac.uk.

structures; F.3.1 [**Logics and Meanings of Programs**]: Specifying and Verifying and Reasoning about Programs

General Terms: Design, Reliability, Verification

Additional Key Words and Phrases: Compositional techniques, concurrency, context constraints, distributed systems, labeled transition systems, reachability analysis, state space reduction, static analysis, validation

1. INTRODUCTION

Distributed systems have been widely deployed for applications from industrial control to commerce. These applications are often critical in nature where an error can have catastrophic consequences. For instance, the first flight of the space shuttle was delayed by a subtle error which was traced to an improbable race condition in the flight control software [Garman 1981]. Software practitioners require sound analysis techniques to help to discover such defects and to assure satisfaction of particular properties. Unfortunately, distributed systems are generally more complex to analyze than their sequential counterparts. Nondeterministic ordering of actions and events greatly increases the number of combinations of behavior. Even for relatively small systems, behavioral analysis is impractical without the support of an effective automated technique.

Most concurrent and distributed systems are designed modularly and refined into a tree of processes. The process tree, referred to as the *compositional hierarchy*, reflects the software structure of the system as conceived by software practitioners. This hierarchical structure can provide a sound and effective means of guiding the process of behavioral analysis. At the leaves of the tree are *primitive* processes implemented in some programming language, such as concurrent C. At the intermediate nodes of the tree are *composite* processes or *subsystems*, composed from processes at the lower-level nodes and leaves. The behavior of a process in the compositional hierarchy can be represented by a *labeled transition system* (*LTS*). The LTS is a state machine with transitions labeled by those activities that the process can perform.

Reachability analysis is a well-known approach to provide automated analysis of finite-state distributed systems [Clarke et al. 1983; Peterson 1981; Taylor 1983b]. It constructs a global LTS that represents the overall system behavior. In traditional reachability analysis techniques, the global LTS is constructed in a single step without exploiting the software structure. Clearly, these techniques do not complement modular-design approaches. Modifications have therefore been proposed to add modularity using various computational models in process algebra [Malhotra et al. 1988; Sabnani et al. 1989; Tai and Koppol 1993; Yeh and Young 1991]. These modifications enable subsystems in the compositional hierarchy to be successively composed and simplified. The modified techniques are commonly known as *compositional reachability analysis* (*CRA*). To differentiate

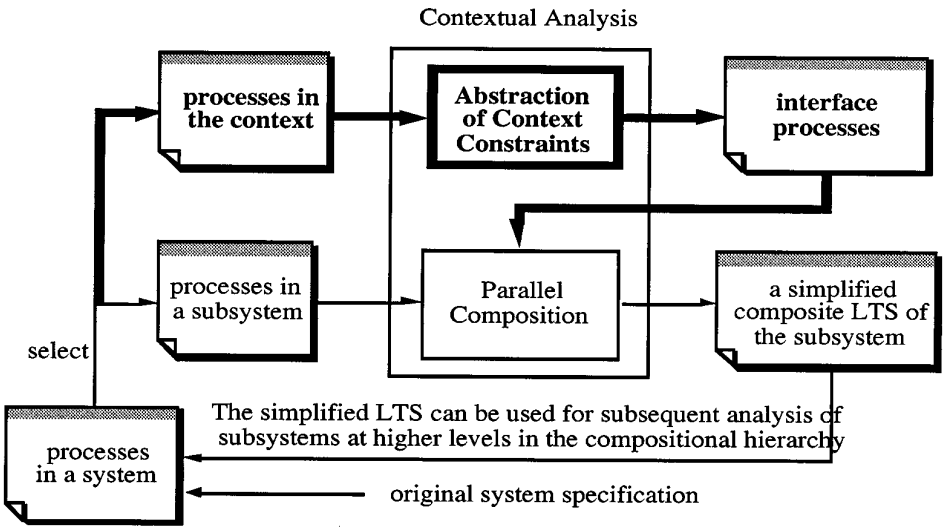

Fig. 1. Method for applying context constraints.

them from the one we propose, they are referred to as *conventional CRA* techniques in later discussions.

The mechanism of "intermediate simplification during composition" in conventional CRA techniques [Malhotra et al. 1988; Sabnani et al. 1989; Tai and Koppol 1993; Yeh and Young 1991] is attractive because it can significantly increase the size of systems that are analyzable with given computer resources [Valmari 1991]. This may not, however, be the case in general. In fact, as we will see, the state explosion problem may even be exacerbated if subsystems are composed without considering their context or environment [Graf and Steffen 1990]. A subsystem thus composed may contain a large number of behavioral traces which are, in fact, not possible given its context. These behavioral traces could be removed by including the context of a subsystem in its composition. It is based on an observation that the context imposes constraints on the subsystem's behavior. These constraints are referred to as *context constraints* [Graf and Steffen 1990] and represented by interface processes in our technique.

Figure 1 presents a schematic diagram showing the method we propose. The bold arrows and boxes show the enhancements made to conventional CRA techniques. First, we select a subsystem to be composed. Context constraints of the subsystem are expressed using interface processes which can be automatically derived and/or specified explicitly, based on application knowledge. These processes can then be composed with the underlying processes of the subsystem to form a simplified composite LTS. The LTS describes the behavior of the subsystem, subject to its context. Software developers may correct the design at this stage without the need for a complete analysis of the whole system. If no correction is necessary, the LTS can be used as a substitute for the original subsystem in subsequent

compositional reachability analysis. We refer to this technique of composing subsystems as *contextual analysis*.

The technique is illustrated in this article using a simple clients/server example implementing a round-robin protocol. As we show in later sections, substantial performance improvement is possible when applying the technique. Section 2 presents background information on the use of context constraints in the analysis of distributed systems and discusses related work. Section 3 introduces the LTS formalism used to model behavior. A description of the clients/server example using the LTS formalism is then given in Section 4. Section 5 discusses conventional CRA techniques and the associated limitations. In Section 6, we present the underlying framework to include context constraints in conventional CRA. Context constraints are captured by interface processes transparent to the distributed system being analyzed. Section 7 outlines an interface construction algorithm which can be used to derive interface processes automatically. Some properties of the contextual CRA technique are given in Section 8, followed by performance figures in Section 9. In Section 10, we discuss limitations of the technique and, in Section 11, present a mechanism which enables software developers to also specify interface processes explicitly. The mechanism identifies erroneous interface processes and locates the error sources. Finally, conclusions are presented in Section 12.

2. BACKGROUND

2.1 Context Constraints

The use of context constraints in CRA was suggested by Larsen and Milner [1987] and Graf and Steffen [1990]. Inclusion of context constraints in the composition of a subsystem will remove, at an early stage of CRA, those behavioral traces forbidden by the context of the subsystem. Early removal of these behavioral traces can substantially reduce the computational costs of analysis. As these forbidden traces are irrelevant to the distributed system being analyzed, their removal also provides a more precise description of the behavior of the subsystem.

For instance, consider a simple job scheduler example originally introduced by Larsen [1989] to illustrate the usefulness of context constraints in software verification. The scheduler consists of three primitive processes A, B, and C as shown in Figure 2.

The behavior of the primitive processes is defined as LTSs in Figure 3, where circles represent states, and arcs represent transitions. The scheduler is defined as the parallel composition $A \parallel B \parallel C$. Shared actions ab, bc, and ca are executed synchronously by two processes. Consider a subsystem BC formed by $B \parallel C$. When B is composed with C (Figure 4), the resultant LTS of BC contains states and transitions forbidden by A. These forbidden states and transitions could be eliminated if the constraints imposed by process A onto B and C were considered in the composition.

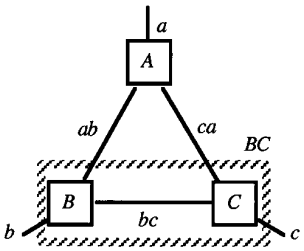

Fig. 2. A connection diagram of three primitive processes A, B, and C.

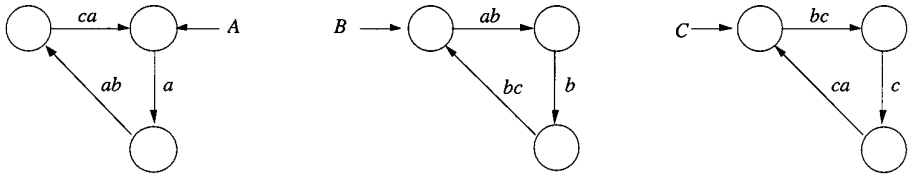

Fig. 3. Behavior of the primitive processes in a simple scheduler.

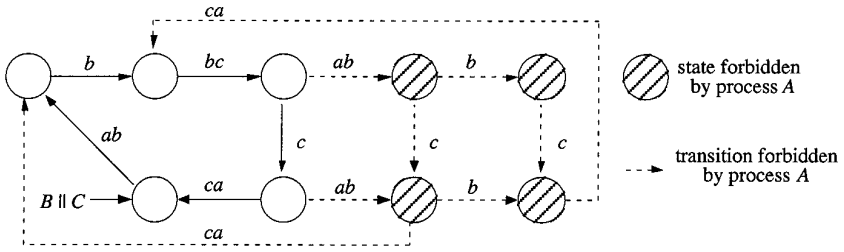

Fig. 4. An LTS formed by parallel composition of processes B and C.

2.2 Related Work

As illustrated, state explosion can be alleviated if context constraints are incorporated in behavior analysis. Larsen and Milner [1987] proposed the use of context constraints (an environment) in parameterizing behavior bisimulation. The proposed bisimulation includes the environment in equating process behavior and is called *relativized bisimulation*. Two processes are considered to be equivalent if they behave the same in a given environment. The environment, given as a set of expressions in process algebra, defines the scope in which process behavior is of interest. The relativized bisimulation can equate more processes than that using the original bisimulation given by Milner et al. [1989]. This allows the behavior of subsystems to be substituted by simpler LTSs and helps to reduce the effort involved in compositional reachability analysis. In the technique, both the environment and the LTS used for substitution are specified by users.

Clarke et al. [1989] proposed a framework for constructing a compositional proof for systems of finite-state processes. In a composite system $P_1 \| P_2$, the behavior constraints imposed on P_1 by P_2 are captured using an interface process I. The interface process is so specified that logical proper-

ties which hold for $P_1 \| I$ also hold for $P_1 \| P_2$. This can be achieved if I behaves equivalently to $P_2 \uparrow \alpha P_1$ where $P_2 \uparrow \alpha P_1$ denotes a process formed by restricting P_2 to the actions in P_1. Let φ and $\equiv$ represent a logical property and behavior equivalence, respectively. The proposed mechanism can be expressed as follows:

$$(I \equiv P_2 \uparrow \alpha P_1 \wedge P_1 \| I \models \varphi) \Rightarrow P_1 \| P_2 \models \varphi.$$

Since P_1 is a subsystem in $P_1 \| P_2$, the mechanism enables logical properties of a system to be verified at subsystem levels, i.e., $P_1 \| I$. Verified subsystem properties hold in conjunction with others at later stages. Interesting applications of the framework have been suggested for the verification of hardware systems. Components in these hardware systems are connected in a regular way such that the interface process of each component generally takes the same form. Thus interface processes can be readily obtained. However, the effectiveness of the method greatly depends on the existence of an efficient algorithm to compute the interface processes, particularly for less regular systems. The formulation of an algorithm for deriving interface processes was not addressed by Clarke et al. [1989]. Further, since the implication operator in the above formula is only from left to right, a weakness of the framework is that the conjunction of properties at the final stage may only represent a partial set of system properties. This occurs where there are some properties that could not be decomposed into subsystem properties, e.g., properties that involve actions in more than one subsystem. The construction of a global LTS has not been addressed in the framework.

Graf and Steffen [1990] presented a theory for the construction of global LTSs from which system properties could be directly verified. The work utilizes context constraints in building composite LTSs for subsystems. These LTSs can be used to build a global LTS which represents all possible behaviors of the distributed system being analyzed. Since the theory focuses on LTS construction rather than on the preservation of logical properties, it is orthogonal to that proposed by Clarke et al. [1989]. Nevertheless Graf and Steffen adopted a similar philosophy to Clarke et al. by expressing context constraints in terms of interface processes. However, since these constraints are provided by users, they might be improperly specified. This is the case if the specified constraints eliminate some legitimate execution sequences of the system. To detect improperly specified constraints, the computational model of LTS was extended. The new model includes an additional primitive providing for *undefined predicates* which are used to model undefined transitions. Constraints are identified to be improperly specified if undefined predicates appear at the global LTS. The global LTS thus constructed does not faithfully represent the behavior of the distributed system.

Yeh [1993] proposed a similar technique to that of Graf and Steffen, but based on the process algebra ACP [Bergstra and Klop 1985]. Also, rather than specifying context constraints as separate interface processes, they

are inserted directly in the primitive processes by means of transitions labeled with the actions *sleep*, *wake*, and *activate*. The insertion is made manually by users. These inserted actions must be properly indexed so that matching actions share the same index (for example, $sleep_i$ matches $wake_i$). A $sleep_i$ transition indicates a point where the process involved can never rendezvous with the environment. Its execution makes the process transit into a *trap* state. These trap states appear in the global LTS when constraints are incorrectly specified. The effect of executing a $sleep_i$ transition can be nullified if it can rendezvous with a matching $wake_i$ transition in another process. The semantics of *sleep* and *wake* enables users to control the enumeration of process states. Sleep transitions can be propagated to the composite processes by means of matching activate transitions. If $sleep_i$ and $activate_i$ are actions that can be executed by processes P and Q respectively, $sleep_i$ would be propagated to $P\|Q$ and becomes executable in $P\|Q$. To accommodate the theory, a new composition operator and class of behavior equivalence called *almost-weakly-bisimulation* were defined. A weakness of the technique is that the new composition operator is no longer associative.

Though very promising, the techniques mentioned above are not commonly used in practice. One of the main reasons is that they have not yet been automated: context constraints are not generated, but need to be specified by the users. The techniques also lack a mechanism which indicates the error sources of improperly specified constraints. In this article we present our approach to *contextual compositional reachability analysis* (*contextual CRA*), which is a refinement of Graf and Steffen's technique for building global LTSs for distributed systems, and which addresses the above problems. First, it supports compositional reachability analysis where interface processes are automatically included at each step. It extends previous work [Cheung and Kramer 1995b], applying it to compositional reachability analysis. Second, it enables users to specify interface processes based on their application knowledge. These interface processes could however be incorrectly specified. To detect these erroneous processes, the state machine model is augmented with a special undefined state [Cheung and Kramer 1995a]. Erroneous interface processes result in the existence of a reachable undefined state in the global LTS. In this case, the technique locates those transitions in the interface processes which cause the errors. Based on this information, users can correct the interface processes and repeat the analysis. In addition, automatically derived interface processes can be combined with user-specified ones if desired.

Provided it is free of reachable undefined states, the technique ensures that the global LTS constructed describes the system behavior faithfully. The global LTS is shown to be observationally equivalent [Milner 1989] to that constructed using conventional CRA. In addition, our technique uses a state machine model similar to that used in conventional CRA. Undefined states are transparent to users because they can be inserted automatically by the technique. Therefore, contextual CRA provides the users with essentially the same view of their designs as conventional CRA. As in the

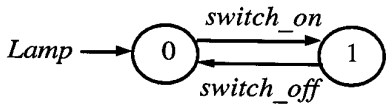

Fig. 5. An LTS description of a lamp switch.

work of Graf and Steffen [1990] and Yeh [1993], our discussion focuses on the construction of composite LTSs; application of context constraints to preserve logical properties as suggested by Clarke et al. [1989] is not addressed. The main theme of the article is to present a simple and promising automated technique for the extraction and application of context constraints in compositional state-space analysis.

3. LABELED TRANSITION SYSTEMS (LTS)

3.1 Description

The behavior of a process in a distributed system can be modeled as a *labeled transition system* (*LTS*). An LTS contains all the states the process may reach and all the transitions it may perform. For instance, Figure 5 represents an LTS describing a lamp which can be either on or off. The lamp can go from state 0 to 1 as the consequence of an external action consisting of pushing the *switch_on* button. Pushing the *switch_off* button causes the opposite transition.

If there is no need to distinguish them, similar activities performed by a system are labeled by the same action. The set of actions which are considered relevant for a particular description of a process is called its *alphabet* [Hoare 1985]. In the above example, the alphabet of *Lamp* is {*switch_on*, *switch_off*}. The alphabet is a permanent predefined property of a process. Logically, a process may only perform actions belonging to its alphabet. For example, *Lamp* cannot perform an action *deliver* because it is outside its alphabet. However, a process might never perform an action in its alphabet. An alphabet is so chosen as to make analysis tractable. This involves abstraction decisions, ignoring many other properties and actions considered to be of lesser interest. A process may perform some activities which cannot be influenced by its environment. These activities are labeled by an internal action which is represented by the symbol τ.

The LTS computational model has been widely used in the literature for specifying and analyzing distributed systems [Clarke et al. 1989; Ghezzi et al. 1991; Kemppainen et al. 1992; Rabinovich 1992; Valmari 1992]. In the model, communicating processes are synchronized through actions sharing the same labels. For example, let a represent the action in which a machine in a flexible manufacturing system transfers a part to a conveyor belt. The action a occurs only if (1) the machine is ready to hand over the part and (2) the conveyor belt is simultaneously prepared to receive the part. Thus the action a requires simultaneous participation of both the processes involved, and a must also be a possible action in the standalone behavior of each process.

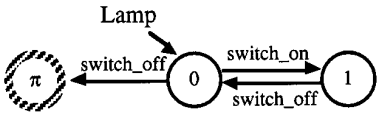

Fig. 6. A process with an undefined state.

LTSs can be analyzed with model-checking tools [Clarke et al. 1986] and compared with other LTSs to verify behavioral equivalence [Hennessy 1988]. They can also be simplified. The simplification of an LTS means the construction of a behaviorally equivalent, but smaller, LTS. The purpose of simplification is to reduce the computational effort in subsequent processing steps.

3.2 Definitions

Definition 3.2.1 gives the formal definition of an LTS. Since there is a one-to-one mapping between a process and its LTS, the terms "process" and "LTS" are used interchangeably in our discussion.

Definition 3.2.1. An LTS of a process P is a quadruple $\langle S, A, \Delta, p \rangle$ where

(1) S is a set of states;
(2) $A = A' \cup \{\tau\}$, where A', denoted as αP, is the communicating *alphabet* of P which does not contain τ;
(3) $\Delta \subseteq (S - \{\pi\}) \times A \times S$ is a mapping from a state and an action onto another state;[1]
(4) p is a state in S to denote the initial state of P.

Here, τ represents an internal action, and π represents an *undefined state* at which the associated process is trapped. When a process is trapped, it becomes *undefined* and cannot participate in further activities. Behavior of an undefined process can be represented as $\Pi = \langle \{\pi\}, \mathcal{A}, \emptyset, \pi \rangle$ where $\mathcal{A}$ denotes the universal set of actions. Figure 6 shows the state diagram of a process with an undefined state. A process is *totally defined* if it contains no reachable undefined states. Typically, all processes specified by users are totally defined. As we will see later in Section 11, undefined states can be inserted automatically by the contextual analysis technique for error checking.

We write $P \xrightarrow{a} P'$ when process $P = \langle S, A, \Delta, p \rangle$ transits into P' after execution of action a. This happens if and only if (p, a, p') is a transition in Δ and p' the initial state of P'. The behavior of P' is given by

$$P' = \begin{cases} \langle S, A, \Delta, p' \rangle & \text{if } p' \neq \pi \\ \Pi & \text{if } p' = \pi \end{cases}$$

[1]This denotes the set subtraction operator.

A *trace* of a process is the sequence of actions (excluding τ) that the process can perform at its initial state. For example, the sequence $\langle switch\_on, switch\_off \rangle$ is a trace in Figure 6. A trace is *undefined* when its execution leads a process to an undefined state. The set of traces exhibited by a process P is commonly written as $tr(P)$. A transition is *reachable* in a state diagram if it emerges from a reachable state; otherwise it is *unreachable*. A transition is *undefined* if it ends at an undefined state, such as (0, *switch_off*, π) in Figure 6.

3.3 Restriction Operator

Observability of actions in a process can be controlled by a restriction operator $\uparrow$. Intuitively, $P \uparrow L$ represents the process projected from P in which only the actions in set L are observable. Rules 3.3.1, 3.3.2, and 3.3.3 give the essential properties of the operator, which is formally defined in Appendix A (Definition A.3).

Rule 3.3.1

$$\frac{P \xrightarrow{a} P'}{P \uparrow L \xrightarrow{a} P' \uparrow L}(a \in L)$$

Rule 3.3.2

$$\frac{P \xrightarrow{a} P'}{P \uparrow L \xrightarrow{\tau} P' \uparrow L}(a \notin L)$$

Rule 3.3.3

$$P = \prod \Leftrightarrow P \uparrow L = \prod$$

3.4 Composition Operator

Processes in a distributed program may be composed by a composition operator $\|$ similar to that used in CSP [Hoare 1985]. $P\|Q$ is the parallel composition of processes P and Q with synchronization of the actions common to both of their alphabets and interleaving of the others. Rules 3.4.1–3.4.4 summarize the essential properties of the operator. Its formal definition can be found in Appendix A (Definition A.4).

Rule 3.4.1

$$\frac{P \xrightarrow{a} P'}{P\|Q \xrightarrow{a} P'\|Q}(a \notin \alpha Q,\ Q \neq \prod)$$

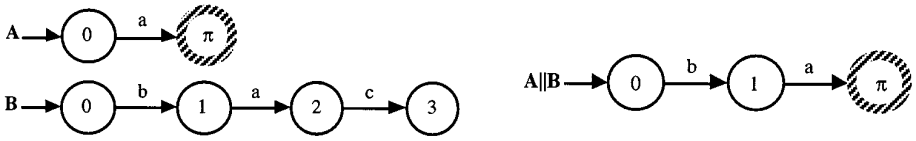

Fig. 7. Processes A, B, and their composite process $A\|B$.

Rule 3.4.2

$$\frac{Q \xrightarrow{a} Q'}{P\|Q \xrightarrow{a} P\|Q'}(a \notin \alpha P,\ P \neq \prod)$$

Rule 3.4.3

$$\frac{P \xrightarrow{a} P'\ \ Q \xrightarrow{a} Q'}{P\|Q \xrightarrow{a} P'\|Q'}(a \in \alpha P \cap \alpha Q)$$

Rule 3.4.4

$$(P = \prod \vee Q = \prod) \Leftrightarrow P\|Q = \prod$$

The composition operator is both commutative and associative. It enforces a *multiway communication* such that if an action a is common to alphabets αP, αQ, and αR it must be executed synchronously by processes P, Q, and R; disjoint actions can be executed asynchronously. Multiway communication is used in CSP [Hoare 1985], whereas two-way communication is adopted in CCS [Milner 1989]. Although there are situations where one is more appropriate than the other, each can be modeled using the other. Figure 7 shows the composite LTS of $A\|B$ composed from processes A and B by following the transition rules. The initial state 0 of $A\|B$ denotes the situation where both A and B are at their own initial state 0. According to the rules above, process A cannot fire the action a at state 0 until B has fired action b and reached state 1. Thus only action b can be initially executed by $A\|B$. After that, a can be executed. But its execution leaves process A, and thus $A\|B$, in an undefined state.

3.5 Behavioral Equivalences

The notion of *observation* on processes is postulated to describe the process behavior conceived by an observer conducting experiments on processes. Behavior equivalence is a concept which identifies a pair of processes which cannot be distinguished by such observations. When process P is equivalent to Q, it means P will do whatever Q does. In other words, we are equally happy to use either process as far as their behavior is concerned. Over the years, there have been many studies of behavior equivalence. Commonly used behavioral equivalences in the literature include trace equivalence [Rabinovich 1992], failure equivalence [Brookes et al. 1984; Hoare 1985],

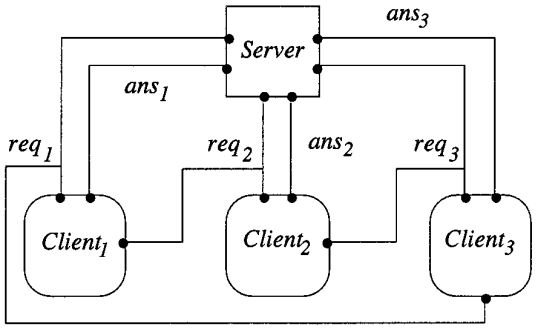

Fig. 8. A connection diagram of the clients/server system when $n = 3$.

testing equivalence [DeNicola and Hennessy 1984], and observational (weak) and strong equivalences [Milner et al. 1989]. Among these, strong and observational equivalences are popularly used to distinguish behaviors of communicating processes, and their definitions are slightly adapted here to suit our state machine model and are referred to as strong and weak semantic equivalences respectively. They include an additional criterion which equates undefined processes to Π. The adaptation is to accommodate both the multiway communication semantics of the composition operator and the inclusion of the undefined state π.

Strong semantic equivalence, denoted as $\sim$, is used to relate two processes whose behaviors are indistinguishable to an observer who is able to observe all actions, including internal actions. *Weak semantic equivalence*, denoted as $\approx$, is used to relate two processes that are observationally indistinguishable where their internal actions are invisible. Strong semantic equivalence is a subset of weak semantic equivalence. Both equivalences assume that external observers can identify processes that have assumed state π. Formal definitions of strong and weak semantic equivalences can be found in Appendix A (Definitions A.5 and A.6).

4. A CLIENTS/SERVER SYSTEM

In order to illustrate our approach, consider a simple clients/server system, *CltSvr*, consisting of a shared server process, *Server*, and n client processes, $Client_1, \ldots, Client_n$. The server holds a shared resource which is accessed exclusively by one client at a time.

There is no central arbiter in the system, and requests from clients are served using a round-robin protocol. Figure 8 shows the overall configuration of the system for $n = 3$ where boxes represent the processes in the system and where directed edges represent the message links between processes. Each client, $Client_i$, requests a service in turn by action req_i. After making a request, a client waits for a round before it makes its next request. The desired effect is achieved by passing on the right to make a new request. For instance, when $Client_1$ has made a request, it uses req_2 between $Client_1$ and $Client_2$ to pass to $Client_2$ the right to make further

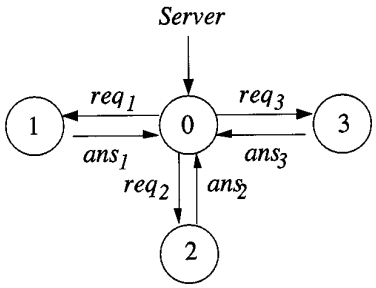

Fig. 9. An LTS describing the behavior of *Server*.

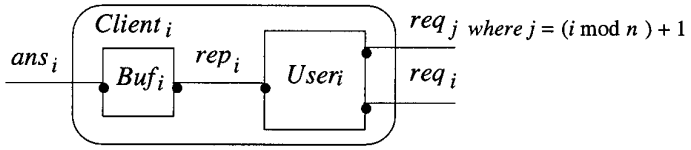

Fig. 10. Connection diagram of a client process.

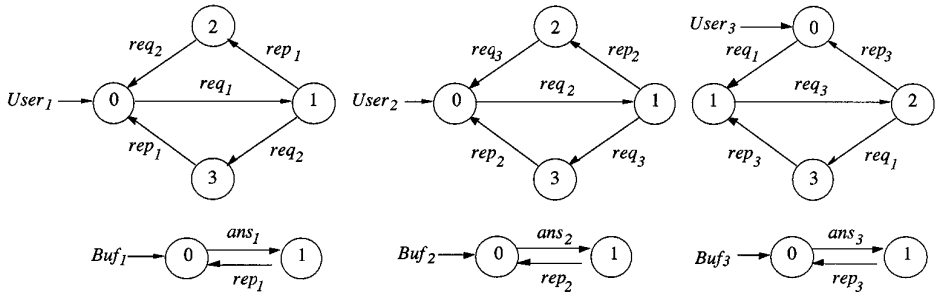

Fig. 11. LTSs describing the behavior of $User_i$ and Buf_i.

requests. On receiving a request, action req_i, *Server* (Figure 9) responds with an answer, action ans_i.

The clients/server system *CltSvr* is specified as $(Server \| Client_1 \| Client_2 \| Client_3)$. Each client process $Client_i$ (Figure 10) consists of a buffer process Buf_i and a user process $User_i$ such that $Client_i = (Buf_i \| User_i) \uparrow \alpha Client_i$, where $\alpha Client_i = (\alpha Buf_i \cup \alpha User_i) - \{rep_i\}$.

Figure 11 shows the behavior of the processes $User_i$ and Buf_i. The system allows overlapping concurrent behavior. A user may read the message from its buffer concurrently with another user making a request. For instance, $User_1$ may read the message from Buf_1 by executing rep_1 while $User_3$ makes a request to the server by executing req_3. As a result, several buffers may simultaneously contain messages. In other words, any combination of empty and full buffers is possible, and the number of states of the clients/server system is exponential in n.

Given such a system, the users may wish to verify automatically if there is any deadlock in the system, if two answers might be delivered for a request, or if the requests are really made in a round-robin manner. As we

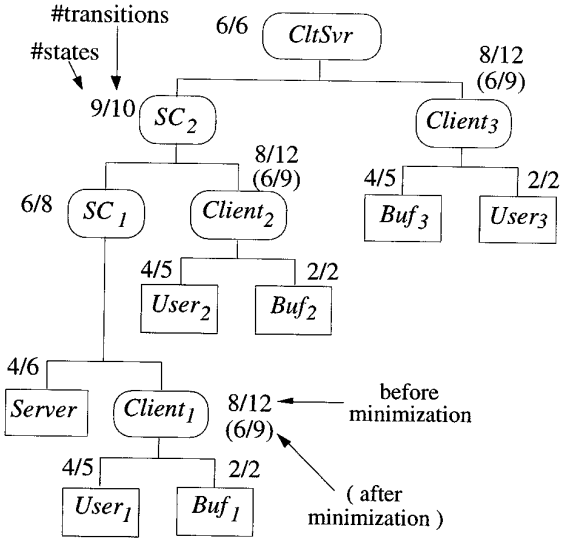

Fig. 12. A compositional hierarchy of a clients/server system for CRA.

will see in Section 5.2, these questions can be answered by traversing the global LTS generated using compositional reachability analysis.

5. CONVENTIONAL CRA

Before discussing the application of context constraints to reachability analysis, let us briefly review the basic concepts of conventional CRA suggested in the literature [Malhotra et al. 1988; Sabnani et al. 1989; Tai and Koppol 1993; Yeh and Young 1991]. Conventional CRA techniques add compositionality to traditional reachability analysis. A major objective is to control the *state explosion problem* [Smolka 1984; Taylor 1983a] which occurs when the state space of a system increases exponentially with the number of its constituent processes. Using a remote temperature sensor system and dining philosopher example, Yeh [1993] demonstrated that adding such a compositionality mechanism to traditional reachability analysis could result in a dramatic decrease in the number of generated states; the performance was comparable to that using constrained expressions [Avrunin et al. 1991]. Sabnani et al. [1989] reported on an experiment applying a conventional CRA technique to the Q.931 protocol [CCITT 1984]. They found that the intermediate state space graphs generated never exceeded 1000 states, even though the global state space graph given by reachability analysis of the protocol contained over 60,000 states.

5.1 Problem Decomposition

The key to conventional CRA is to decompose a system into a hierarchy of small subsystems which are then successively composed and simplified. For example, the software developer of the clients/server system may choose to decompose the system as shown in Figure 12. The rounded boxes represent

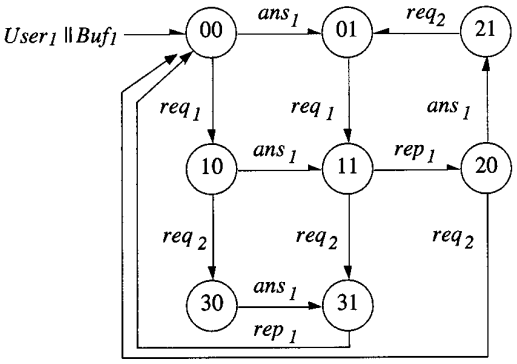

Fig. 13. The composite LTS of $User_i \| Buf_i$.

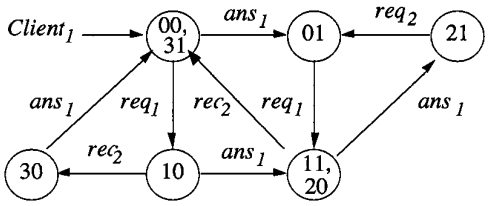

Fig. 14. The simplified LTS of $Client_1$ after the hiding of internal action rep_1.

subsystems, and the square boxes represent primitive processes specified as LTSs.

5.2 Analysis

The composite LTS of a subsystem is constructed by composing the LTSs of its constituent processes. For instance, the composite LTS (Figure 13) of $Client_1$ can be given by combining the LTSs of $User_1$ and Buf_1. Each state is labeled by two numbers: the first gives the current state of $User_1$ and the second that of Buf_1.

In minimizing a subsystem, internal actions which are not observable can be abstracted away from its composite LTS. For instance, action rep_1 is an internal action in subsystem $Client_1$ and, therefore, can be abstracted away from its composite LTS in Figure 13. This LTS is then simplified to reduce the number of states while at the same time respecting weak semantic equivalence. The simplification results in a simpler LTS (Figure 14) describing a behavior observationally indistinguishable from that of $Client_1$.

This simplified LTS can then be used as a substitute for the behavior of $Client_1$ in the subsequent steps of conventional CRA. This process of intermediate composition and simplification of subsystems continues until the global LTS (Figure 15) of the system is obtained. The sizes of the LTSs for intermediate subsystems generated by CRA are given in Figure 12. In the figure, the subsystem sizes after simplification are given in brackets if they are different from those before simplification.

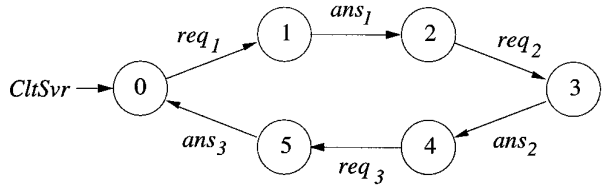

Fig. 15. The global LTS modeling the behavior of the clients/server system.

Observable system behavior such as safety and liveness properties can be examined by traversing the global LTS. For example, by traversing the global LTS in Figure 15 we can verify that the clients/server system possesses the following properties:

—it is free from deadlock because all states have outgoing transitions;
—it never delivers two answers in response to a single request because req_1 and ans_1 are alternating actions, as are req_2 and ans_2, req_3 and ans_3; and
—requests are made by clients in a round-robin manner from $Client_1$ to $Client_3$ because req_1, req_2, and req_3 must occur in sequence.

Liveness properties expressed in, say, temporal logics can be verified using automatic model checking by traversing the global LTS [Clarke et al. 1986].

5.3 Limitations

In general, conventional CRA may not always be able to control the problem of state explosion [Graf and Steffen 1990]. On the contrary, the problem may even be exacerbated. In conventional CRA, subsystems are locally simplified without including their contexts or local environment. A subsystem may therefore contain many transitions and states which are forbidden in the context of the whole system. For instance, the shaded states and dashed transitions in the simplified LTS of $Client_1$ (Figure 16) are forbidden within the global context of the clients/server system. These states and transitions are never executed by $Client_1$ because of the context constraints imposed by its neighbors: $Client_2$, $Client_3$, and the *Server*. For example, when $Client_1$ starts at state 0, it cannot execute the action ans_1, as it would violate a constraint imposed by the *Server* (Figure 9) which requires that the number of occurrences of ans_1 cannot exceed that of req_1. When $Client_1$ reaches state 1 after executing req_1, it cannot execute req_2 because the *Server* does not accept req_2 before it provides an answer, ans_1, to $Client_1$. After $Client_1$ has reached state 2, it cannot execute ans_1 without another occurrence of req_1. According to the behavior of $User_1$ in Figure 11, req_1 has to be preceded by an execution of req_2.

The forbidden states and transitions in Figure 16 can complicate our understanding of the precise behavior of $Client_1$. Although the behavior traces introduced by these states and transitions are eliminated at some later stage in conventional CRA, their existence often requires substantial computational resources and aggravates the state explosion problem. Fur-

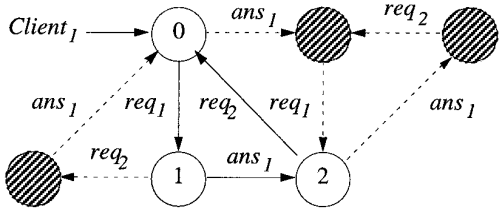

Fig. 16. Forbidden states and transitions of $Client_1$.

thermore, the number of forbidden states and transitions can grow larger when constructing an LTS from smaller subsystems that contain forbidden states and transitions. The growth normally stops when the subsystem being composed contains most of the processes in the overall system. As we will see in Section 9, experimental results show that the size of the largest intermediate LTS for our clients/server system grows exponentially as the number of clients in the system increases.

In addition, conventional CRA techniques are sensitive to the structures of the compositional hierarchy. A small difference in the compositional hierarchy may lead to a large difference in the computational effort. For example, the clients/server system could be decomposed into the compositional hierarchy shown in Figure 17. The size of the largest subsystem *C123* contains 81 states and 216 transitions; this is even larger than that generated directly by traditional reachability analysis which contains 36 states and 72 transitions. Again, the state explosion problem becomes more significant as the number of clients increases.

Figure 18 shows the sizes of the largest simplified LTSs generated by CRA based on a compositional hierarchy similar to those in Figures 12 and 17 respectively when the system contains six clients.

In the following sections we discuss how CRA may be enhanced so that the above problems can be substantially alleviated. The enhancement is based on a refinement of the work by Graf and Steffen [1990]. The refinement made is to simplify the computational model, streamline the application of context constraints, and provide a concrete technique to derive context constraints automatically.

6. INCLUSION OF CONTEXT CONSTRAINTS

The globally forbidden behavior traces can be removed from the LTS of $Client_1$ if context constraints are included during the construction of the composite LTS. However, inclusion of context constraints should not jeopardize the correctness of the analysis. That is, the global LTS constructed with the context constraints should be weakly semantically equivalent to that given by conventional compositional reachability analysis.

6.1 Approach

Let us consider a distributed program $P = P_1 \parallel \ldots \parallel P_n$ and suppose we wish to construct an LTS for P_1. The *context* of P_1 is given by $P_2 \parallel \ldots \parallel P_n$.

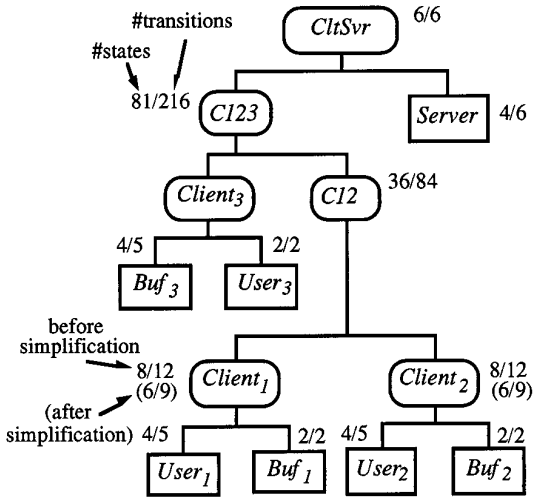

Fig. 17. An alternative compositional hierarchy of the clients/server system.

Conventional CRA based on a hierarchy similar to that in Figure 12		Conventional CRA based on a hierarchy similar to that in Figure 17		Traditional Reachability Analysis	
#states	#transitions	#states	#transitions	#states	#transitions
55	72	7,776	37,584	576	2,016

Fig. 18. The largest subsystems generated by CRA and reachability analysis.

In order to apply context constraints in the LTS construction, we need to find a transformation function Ψ_P that satisfies the following three requirements:

Requirement 6.1.1

(1) $\Psi_P(P_1)$ results in an LTS which describes a more restricted behavior than that of P_1;
(2) $\Psi_P(P_1)$ includes the context constraints of P_1;
(3) P is equivalent to a system P' where $P' = \Psi_P(P_1)\|P_2\| \ldots \|P_n$.

Thus, $\Psi_P(P_1)$ describes the behavior of P_1 under the influence of other processes $P_2 \ldots, P_n$. The result of applying CRA to P' is indistinguishable from analysis using the original system P. Since the CRA in our framework is only concerned with observable behavior, the use of weak semantic equivalence is sufficient in requirement (3). It is, however, not sufficient if the formulation of Ψ_P is to be applied to other frameworks in the literature where testing or failure-divergence equivalences are more appropriate. To make the formulation general, strong semantic equivalence is used in requirement (3). Strong equivalence is considered to be the finest equivalence model in concurrency theory [Rabinovich 1992]. Therefore if two

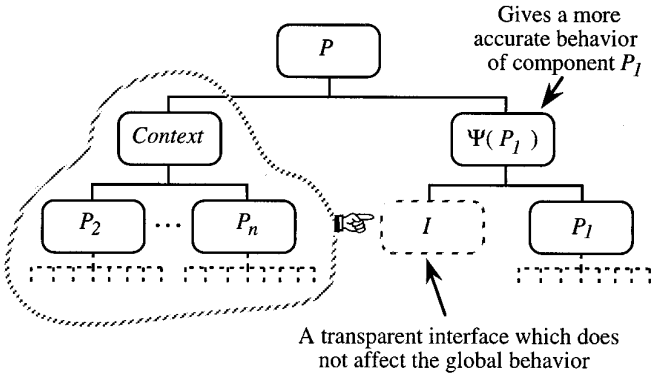

Fig. 19. Inclusion of context constraints using an interface process I.

processes are indistinguishable under strong semantic equivalence, they are also indistinguishable under other commonly used equivalences such as trace, testing, failure, observational equivalence, or weak semantic equivalence.

With the criteria stated in Requirement 6.1.1, a natural choice for Ψ_P is to employ the composition operator $\|$ such that we have the following:

Requirement 6.1.2. $\Psi_P(P_1) = P_1 \| I$ and $\alpha I \subseteq \alpha P_1$, where I is a process capturing the context constraints of P_1 as shown in Figure 19.

6.2 Backward Projection

In Graf and Steffen [1990], the transformation involves application of a backward projection ϑ after the parallel composition. The resultant LTS given by $\vartheta(P_1\|I)$ contains only states and transitions in P_1. The projection ϑ eliminates from P_1 those states and transitions that do not contribute to any reachable states or transitions in $P_1\|I$. Let S and Δ respectively denote the set of states and transitions retained after elimimation. They can be evaluated using the following formulae:

$S = \{s \mid (s, t)$ is a reachable state in $P_1\|I\}$;
$\Delta = \{(s, a, s') \mid ((s, t), a, (s', t'))$ is a reachable transition in $P_1\|I\}$.

The computation of S and Δ requires minimal computational costs, as it can be performed in time proportional to the number of states in $P_1\|I$. Utilization of backward projection guarantees a useful theoretical result—the resultant LTS of $\vartheta(P_1\|I)$ never contains more states than the original process P_1. However, the LTS thus constructed may reinstate forbidden behavior traces of P_1 that have been identified in $P_1\|I$. This can jeopardize the effectiveness of the contextual CRA technique. For example, *Server′* (Figure 20) gives the actual behavior of the *Server* in the context of the clients/server system. The LTS of *Server′* is given by *Server*$\|I$, where I can be derived by the interface construction algorithm to be discussed in Section 7. However, the backward projection of *Server′* onto *Server* would result in an LTS mirroring the *Server* itself. Although *Server′* contains

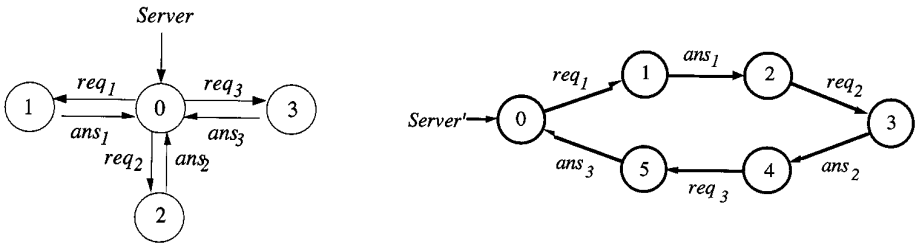

Fig. 20. LTSs of the *Server* before and after transformation.

more states than *Server*, it conveys a more accurate picture of the actual behavior of *Server*. A subsystem given by $Server' \| Client_1$ (six states and six transitions) contains fewer transitions than that given by $Server \| Client_1$ (six states and eight transitions).

The computation of $P_1 \| I$ can be obtained by composing I directly with the components in P_1. However, this is not applicable if backward projection is needed. The projection requires explicit construction of an LTS for P_1. If P_1 is a subsystem, this incurs additional computational costs in the analysis. As a result, the utilization of backward projection may incur higher computational costs. Our experience with contextual CRA suggests that it is generally more effective if backward projection is omitted. For this reason, backward projection is left out in the later discussions of contextual CRA. Note that a mixed strategy could still be adopted in our technique in order to ensure the above-mentioned theoretical result limiting the number of resultant states. For example, we could use the simplified LTS of $P_1 \| I$ in contextual CRA if it contains fewer states than P_1, and otherwise we could use the resultant LTS given by the backward projection.

6.3 The Transparency Property of Interface Processes

Let I be an *interface* process capturing the context constraints of P_1 in P. It is clear that requirements (1) and (2) in Requirement 6.1.1 can be satisfied if $\Psi_P(P_1)$ is chosen to be $(P_1 \| I)$, where αI is a subset of αP. The satisfaction of requirement (3) demands $(P_1 \| P_2 \| \ldots \| P_n)$ to be strongly semantically equivalent to $(P_1 \| I \| P_2 \| \ldots \| P_n)$. In other words, P and $(P \| I)$ must be in strong semantic equivalence. If this is true, we say that I is *transparent* to P. The transparency property ensures that P_1 can be replaced by $(P_1 \| I)$ without affecting the global system behavior. As mentioned, explicit construction of an LTS for P_1 is not required when it is a subsystem. The interface I can be composed directly with the components of P_1 because αI is a subset of αP. For instance, let *Ifc* be a transparent interface capturing the context constraints of $Client_1$. The contextual behavior of $Client_1$ is given by $Client_1 \| Ifc$. Since $Client_1$ is a subsystem, *Ifc* can be composed directly with its components. As a result, the contextual behavior of $Client_1$ is given by

$$Context\_Client_1 = (Buf_1 \| User_1 \| Ifc) \uparrow \alpha Client_1 .$$

$Client_1$ can be substituted by $Context\_Client_1$ in the compositional reachability analysis process. While it is theoretically possible to check the equivalence between the original system and that resulting after the introduction of an interface, this approach is not satisfactory, since the effort required to compute the bisimulation and to find a proper interface could offset the computational costs saved by contextual analysis. A preferable alternative is to identify the criteria which guarantee that an interface is always transparent to the overall system.

6.4 An Interface Theorem

THEOREM 6.4.1 (INTERFACE THEOREM). *Suppose P and I are two finite-state processes which are totally defined, and $\sim$ denotes the strong semantic equivalence relation. Then $P \sim (P \| I)$ if*

(1) $\alpha I \subseteq \alpha P$;

(2) $tr(P \uparrow \alpha I) \subseteq tr(I)$; and

(3) I is a deterministic[2] process free of internal action τ.

The criteria for a transparent interface I with respect to a program P are stated in the *interface theorem*. Proof of the theorem can be found in Appendix B. The first criterion ensures that P and $P \| I$ have the same alphabet, i.e., I does not introduce any new actions. The second criterion ensures that any traces performed by P can also be performed by $P \| I$, i.e., I does not eliminate any legal traces. The third criterion requires the interface I to be a deterministic process (i.e., I does not introduce further nondeterminism) which contains no state transition caused by the internal action τ. Since I contains no internal action, it introduces no new internal transitions into P in the parallel composition. This is crucial in establishing strong semantic equivalence between P and $P \| I$ because the equivalence is sensitive to internal transitions.

In particular, internal action τ is excluded from I because context constraints cannot be captured by transitions labeled with τ. Furthermore, such τ actions may introduce divergence [Hoare 1985] into the system. If the last criterion is relaxed by allowing τ in I, the equivalence relation in the interface theorem becomes weak semantic equivalence. The requirement of I being deterministic can be omitted if backward projection is employed.

6.5 Application of the Interface Theorem

Let us consider a system $P = U \| V$, and assume that we would like to specify an interface I for contextual analysis of U. In this system, V forms the context of U. The global system behavior would not be altered by the introduction of I if I is transparent to V (i.e., $V \sim V \| I$). This is because "$V \sim V \| I$" guarantees that P is strongly semantically equivalent to $P \| I$. By Requirement 6.1.2, αI is a subset of αU. Combining this with the interface

[2]The definition of a deterministic process can be found in Appendix A (Definition A.2).

theorem, we see that I is so constructed as to satisfy (1) $\alpha I \subseteq \alpha U \cap \alpha V$, (2) $tr(V \uparrow \alpha I) \subseteq tr(I)$, and (3) I is a deterministic process free of internal action τ. By doing this, there is no need to check explicitly for equivalence between P and $P\|I$.

For instance in the clients/server example, $Env = (Server\|Client_2\|Client_3)$ forms the context for $Client_1$. Hence, if Ifc is specified so as to satisfy (1) $\alpha Ifc \subseteq \alpha Client_1 \cap \alpha Env$, (2) $tr(Env \uparrow \alpha Ifc) \subseteq tr(Ifc)$, and (3) Ifc is a deterministic process free of internal actions, then Ifc is transparent to Env (i.e., $Env \sim Env\|Ifc$) and $CltSvr$ (i.e., $CltSvr \sim CltSvr\|Ifc$).

An advantage of having transparent interfaces is that they can be composed together forming a stricter interface. Suppose I_1 and I_2 are two interfaces transparent to the context V, then they can be composed together forming a new interface I, which is also transparent to V. This is because "$V \sim V\|I_1$" and "$V \sim V\|I_2$" guarantee that "$V \sim V\|(I_1\|I_2)$." This new interface I would contain all context constraints captured by both I_1 and I_2. This property of superposition is particularly useful if more than one interface can be obtained by different means. For example, I_1 could be an interface derived by an algorithm while I_2 could be an interface specified by users who have knowledge of the behavior of the context of V [Cheung and Kramer 1995a].

7. INTERFACE CONSTRUCTION ALGORITHM

In this section, we present an algorithm that can be used to derive interface processes automatically during compositional reachability analysis. We note that the criteria in the interface theorem are not necessarily met by an interface equal to an LTS of the context. This is because an LTS of the context may be nondeterministic and may contain internal actions. Moreover this approach is not desirable, as it requires the explicit construction of an LTS for the context; the computational effort to compute such an LTS can increase geometrically with the number of processes in the context. Therefore, to be effective, analysis should be supported by an algorithm which is able to construct an interface without the need for an LTS of a given context.

7.1 Description

The algorithm includes in the context only those actions which can constrain the target process. It does this by retaining the actions of those primitive processes from the context which share actions with the target process. The target process could be either a primitive or composite process. All primitive processes in the system are assumed to be totally defined. Let us consider the construction of an interface I for contextual analysis of process U in an application $P = U\|V$, where V is the context of U in P. The interface I can be constructed by the following two steps.

(1) *Reducing Primitive Processes with respect to αU.* Mark those primitive processes of V which share actions with αU in their LTSs. For each

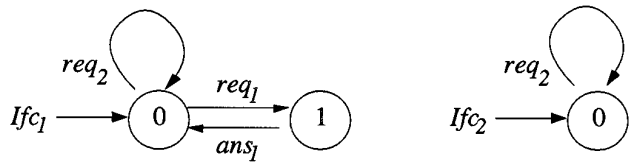

Fig. 21. Reduced LTSs derived from *Server* (left) and $User_2$ (right).

marked process, construct a reduced LTS by repeating the following two operations until no transition remains to be deleted:

(a) delete a transition (s, a, s') where $a \notin \alpha U$;

(b) merge states s and s' together forming a single state.

As a result, each reduced LTS retains only those transitions with actions found in αU. Note that these two operations would remove all internal transitions, if any.

(2) *Transforming Nondeterministic Reduced LTSs into Deterministic LTSs*. Nondeterministic reduced LTSs obtained in step (1) can be converted into deterministic LTSs using a standard algorithm that converts nondeterministic finite automata into deterministic finite automata [Hopcroft and Ullman 1979]. Each deterministic LTS can then be simplified by a minimization algorithm for Deterministic Finite Automata [Hopcroft and Ullman 1979].

The interface I is then given by a parallel composition of the simplified deterministic LTSs obtained in step (2). Note that I satisfies (1) $\alpha I \subseteq \alpha U \cap \alpha V$, (2) $tr(V \uparrow \alpha I) \subseteq tr(I)$, (3) I is a deterministic process free of internal action τ, and I is totally defined, since (4) $I \neq \Pi$. Hence I is transparent to both V and P, i.e., $V \sim V \| I$ and $P \sim P \| I$. The algorithm does not require a derivation of an explicit LTS for V in constructing the interface I. The correctness and complexity of this algorithm are discussed later in Section 7.3.

7.2 Example

We now demonstrate the algorithm by construction of transparent interfaces for contextual analysis of $Client_1$ in the clients/server example.

$$
\begin{aligned}
CltSvr &= CltSvr' \uparrow (\alpha Client_2 \cup \alpha Client_3) \\
CltSvr' &= Client_1 \| Env \\
Env &= Server \| (Buf_2 \| User_2) \| (Buf_3 \| User_3)
\end{aligned}
$$

Only processes *Server* and $User_2$ in *Env* are identified as sharing some actions with $\alpha Client_1 = \{req_1, ans_1, req_2\}$. Therefore two reduced LTSs are constructed in step (1) of the algorithm. To construct the reduced LTS from *Server*, state 0 is merged with 2 and 3 in *Server* (Figure 20) because actions req_3, ans_2, and ans_3 are not in $\alpha Client_1$. After merging, we get a reduced LTS Ifc_1 as shown in Figure 21. The reduced LTS derived from $User_2$ is given as Ifc_2 in Figure 21.

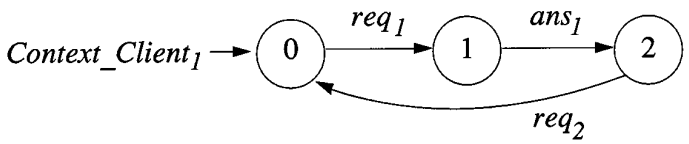

Fig. 22. The composite LTS of $Client_1$ with the inclusion of context constraints.

Execution of step (2) of the algorithm is not required because Ifc_1 and Ifc_2 are already deterministic in their minimal form. Therefore, the interface for contextual analysis of $Client_1$ is given by a composition of Ifc_1 and Ifc_2, i.e., $Ifc_1 \parallel Ifc_2$. The contextual behavior of $Client_1$ can then be composed using the following formula. The resultant LTS after composition and simplification is shown in Figure 22.

$$Context\_Client_1 = (Buf_1 \| User_1 \| Ifc_1 \| Ifc_2) \uparrow \alpha Client_1$$

This composite LTS does not exhibit the forbidden behaviors shown in Figure 16. Compared with the subsystem obtained using conventional CRA (Figure 14), Figure 22 presents a more precise description of the behavior of $Client_1$ in the clients/server system. The composite LTS can be used as a substitute for subsystem $Client_1$ in the subsequent steps of contextual CRA. By the interface theorem, inclusion of the interface processes Ifc_1 and Ifc_2 does not affect the global behavior of system *CltSvr'* and thus *CltSvr*. The global LTS subsequently derived would be weakly semantically equivalent to that obtained by conventional CRA.

7.3 Complexity and Correctness

For a system $P = U \| V$, the complexity of step (1) is at most linear in the size of V. The simplification of each deterministic LTS in step (2) can be computed in $O(N \log N)$ where N is the number of states in the LTS. Therefore the complexity of the algorithm is dominated by the procedure for converting nondeterministic automata to deterministic automata in step (2). The theoretical complexity of the transformation is exponential to the number of states in a nondeterministic reduced LTS. The number of states in an LTS reduced from a primitive process depends on the number of actions it shares with process U. In modular system design, subsystems are often loosely coupled, and therefore the number of shared actions is small. We have applied the algorithm to the clients/server system, a pump control system, and a distributed track control system [Cheung and Kramer 1994a]. We have found that the reduced LTSs were often deterministic or near deterministic and that the computational effort of converting all the reduced LTSs to deterministic automata was usually small.

To show the correctness of the algorithm, let us assume I to be an interface constructed by the algorithm for a system $P = U \| V$, where U is the process to be contextually analyzed and where V is the context of U. I clearly satisfies the first and the third criteria for being transparent to V in the interface theorem.

So let us concentrate on the second criterion, and denote X_1' as a deterministic reduced LTS of a primitive process X_1 in V. It is clear that $tr(X_1 \uparrow \alpha X_1') \subseteq tr(X_1')$. Since X_1 is a constituent process of V, by Lemma B.5 (Appendix B), we have

$$
\begin{aligned}
& tr(\mathrm{V} \uparrow \alpha X_1) \subseteq tr(X_1) \\
\Rightarrow\ & tr((\mathrm{V} \uparrow \alpha X_1) \uparrow \alpha X_1') \subseteq tr((X_1 \uparrow \alpha X_1') \\
\Rightarrow\ & tr(\mathrm{V} \uparrow \alpha X_1') \subseteq tr((X_1 \uparrow \alpha X_1') \text{ (by Lemma B.2 (Appendix B) and } \alpha X_1' \subseteq \alpha X_1) \\
\Rightarrow\ & tr((\mathrm{V} \uparrow \alpha X_1') \subseteq tr(X_1')
\end{aligned}
$$

Similarly, if X_2' is a reduced LTS of a primitive process X_2 in V, then $tr(V \uparrow \alpha X_2') \subseteq tr(X_2')$. From these, we can conclude that $tr(V \uparrow (\alpha X_1' \cup \alpha X_2')) \subseteq tr(X_1' \| X_2')$ by Lemma B.6 (Appendix B). Continuing similar arguments for all reduced LTSs in the interface I, we eventually arrive at the conclusion that $tr(V \uparrow \alpha I) \subseteq tr(I)$.

8. INDEPENDENT ANALYSIS OF DISJOINT PROCESSES

As in conventional CRA, analysis of disjoint[3] processes can be conducted separately in the contextual CRA technique. This is stated in Lemma 8.1. The proof can be found in Appendix B.

LEMMA 8.1. *In a system $P = U \| V \| R$, let U' and V' be two processes describing the contextual behavior of U and V when they separately interact with R. Then $P \approx U' \| V' \| R$.*

The sizes of subsystems given in Figures 12 and 17 are obtained when all disjoint subsystems are analyzed independently and where only the behaviors of primitive processes are accessible in each analysis. The property of independent analysis enables users to expedite contextual CRA by analysis. The property of independent analysis enables users to expedite contextual CRA by analyzing disjoint subsystems in parallel on different machines. The property also facilitates separate development of disjoint processes in a distributed environment. To illustrate this, let us consider a program $P = U \| V \| R$. Suppose that program P is partially specified with only the behavior of process R given. The program is to be completed by two programmers P_1 and P_2 who are responsible for specifying processes U and V respectively. In this scenario, programmer P_1 can assume context R in the design of U. An interface I_u can be derived using the interface construction algorithm such that I_u is transparent to R. Note that this implies I_u is also transparent to P even though the behavior of V is unknown. After U has been specified, an LTS U' can be constructed from $U \| I_u$. Similarly, an LTS V' can be independently constructed by programmer P_2. After that, U' can be combined with V' for the behavior analysis of P as if $P = U' \| V' \| R$.

For example, consider two disjoint subsystems $Client_1$ and $Client_2$ in Figure 17. Suppose that the behavior of only *Server* is accessible during

[3]Two processes are disjoint if they are composed of two disjoint sets of processes.

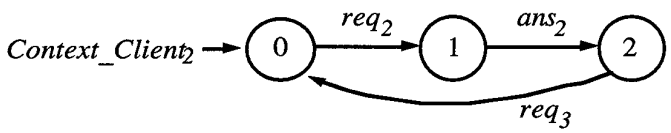

Fig. 23. The composite LTS of $Client_2$ with the inclusion of context constraints.

contextual analysis of $Client_1$ and $Client_2$. Given this, the LTSs constructed from independent contextual analysis of $Client_1$ and $Client_2$ are given in Figures 22 and 23 respectively. These LTSs can be used to replace subsystems $Client_1$ and $Client_2$ respectively for subsequent contextual CRA.

9. PERFORMANCE

A prototype has been developed to support three analysis techniques: global reachability analysis [Holzmann 1991; Taylor 1983b], conventional CRA [Sabnani et al. 1989; Yeh and Young 1991], and the contextual CRA technique described. In Sections 9.1 and 9.2, we compare the performance of the three techniques against different numbers of clients in two variations of the clients/server system. In the first variation, the clients/server system is decomposed into a hierarchy as shown in Figure 12. In the second variation, the clients/server system is decomposed into a hierarchy shown in Figure 17. These variations allow us to gain some insight into the performance of each technique under different compositional hierarchies. Since compositional hierarchy is not used by traditional reachability analysis, its performance is the same in both variations.

9.1 Analysis of Clients/Server System (1)

Figure 24 compares the corresponding performance of the three techniques when the prototype is run on a Sun Sparc/10 workstation. The diagrams compare the size of the largest LTS constructed and the total computational time required for each analysis for the hierarchy shown in Figure 12.

As shown in Figure 24, the state explosion problem encountered in the traditional reachability analysis can be retarded by using conventional CRA techniques in this variation. However, the computational effort required by conventional CRA techniques still grows exponentially with the increase in the number of clients. This problem vanishes with the use of contextual CRA, and the computational effort grows linearly with the increase in the number of clients. When the system is small and contains less than five clients, the performance of contextual CRA closely follows that of conventional CRA. When the system becomes large and contains more than five clients, contextual CRA offers substantially better control of the state explosion problem and outperforms conventional CRA.

9.2 Analysis of Clients/Server System (2)

Figure 25 compares the corresponding performance of the three analysis techniques on a Sun Sparc/10 workstation for the hierarchy shown in Figure 17. The performance of the contextual CRA technique is shown

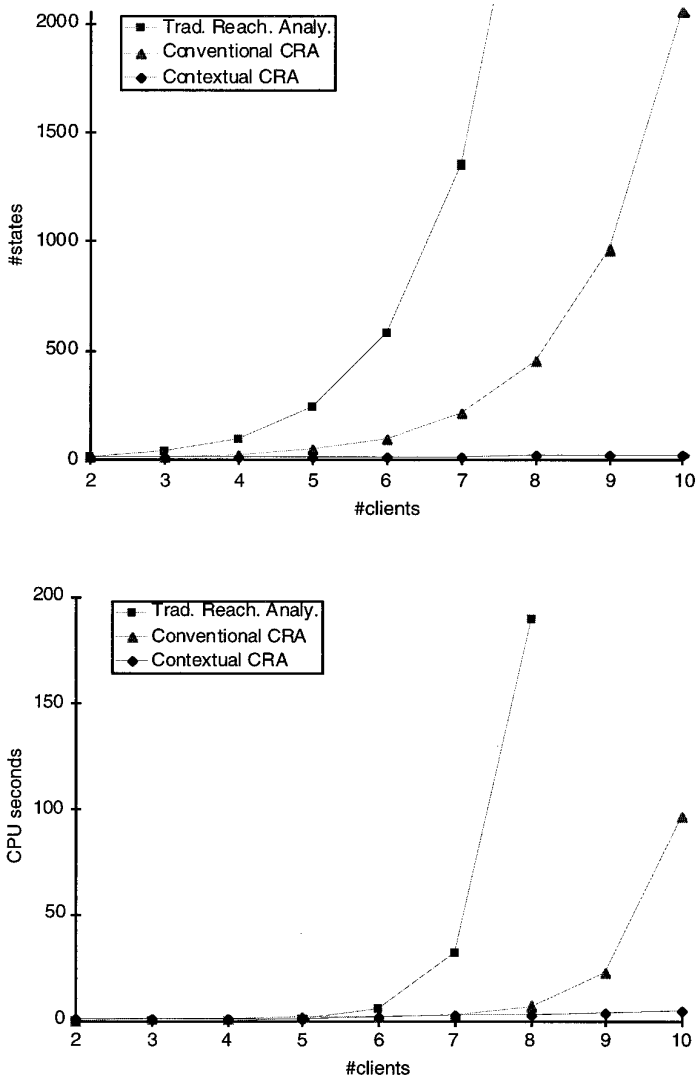

Fig. 24. Performance comparison of different automated analysis techniques; (top) size of the largest LTS constructed; (bottom) total computation time in seconds.

when disjoint subsystems are independently analyzed. The existing prototype fails in the case of conventional CRA with a system containing more than six clients, due to exhaustion of swap space in the workstation.

In this variation, the state explosion problem encountered in traditional reachability analysis is exacerbated by conventional CRA. The largest subsystem generated by conventional CRA is significantly greater than that generated using traditional reachability analysis. The result supports Sabnani et al. [1989] that the performance of conventional CRA is sensitive to the compositional hierarchy and the assertion by Graf and Steffen [1990] that the state explosion problem can be exacerbated by conventional CRA.

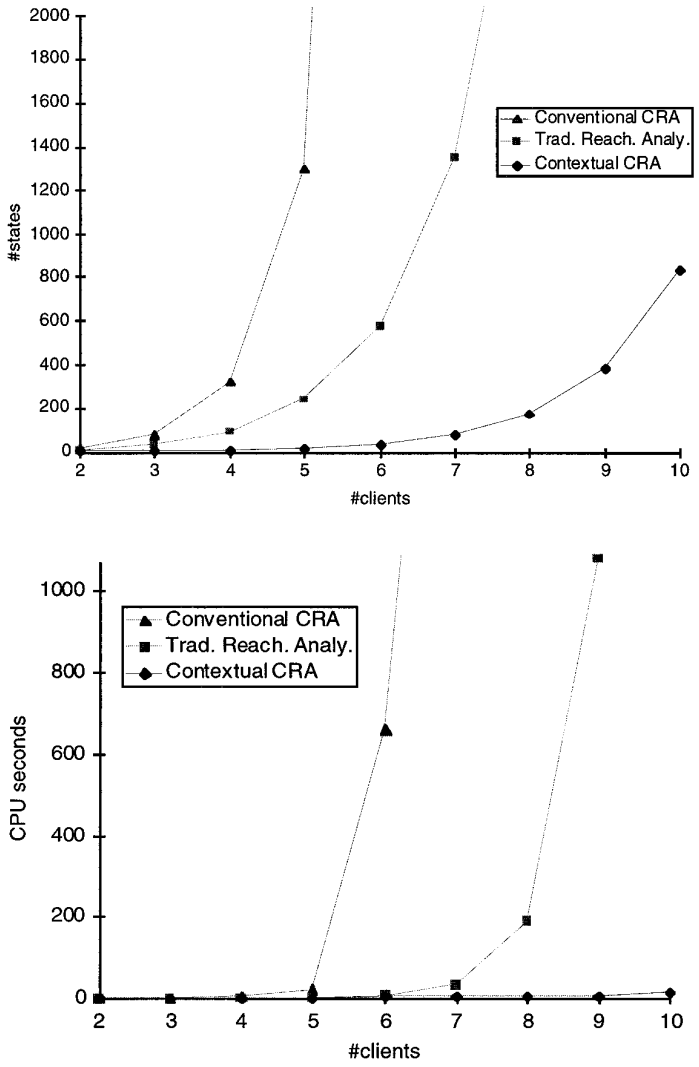

Fig. 25. Performance comparison of different automated analysis techniques; (top) size of the largest LTS constructed; (bottom) total computation time in seconds.

The limitation of conventional CRA can be reduced using contextual CRA, which includes the context constraints. Although contextual CRA still suffers from state explosion when disjoint subsystems are independently analyzed, it outperforms the traditional reachability analysis technique and conventional CRA (Figure 25).

9.3 A Gas Station Example

Contextual CRA was applied to a gas station example originally proposed by Helmbold and Luckham [1985]. The example models an automated gas station with an operator, a pump, a number of customers, and a queue

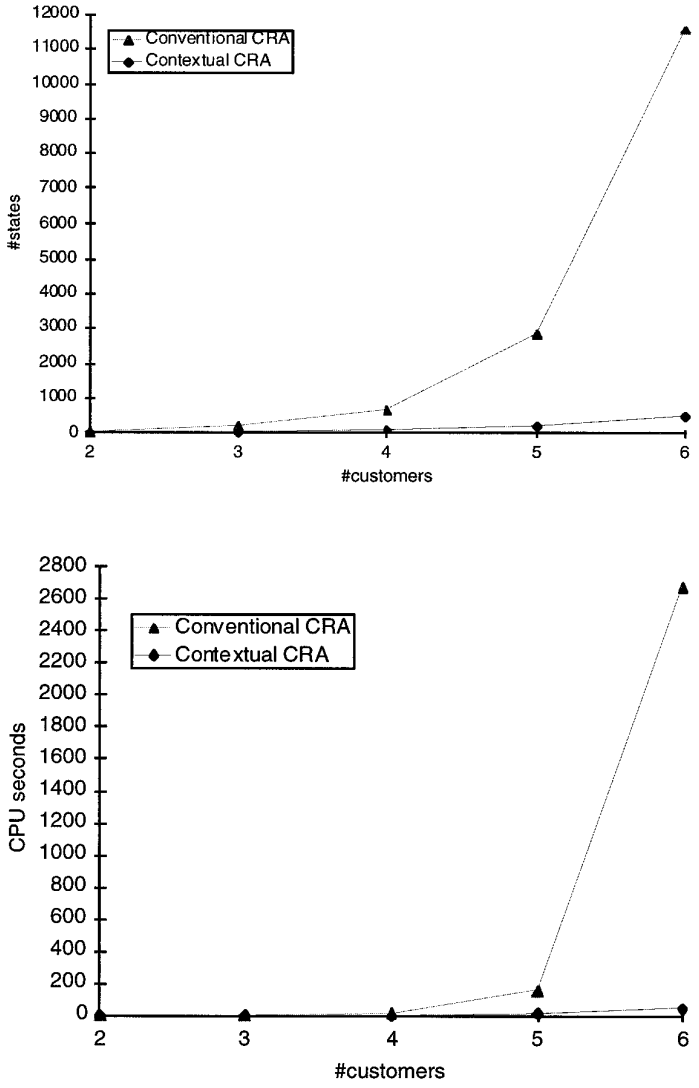

Fig. 26. Performance comparison of conventional and contextual CRA for gas station examples.

holding customers' requests. The specification of the system with two customers can be found in Appendix C. Figure 26 compares the performance of conventional and contextual CRA on a Sun Sparc/10 workstation to analyze gas station systems with different numbers of customers. The size of the largest subsystem for a gas system with six customers can be reduced by 22 times by the inclusion of context constraints.

9.4 A Distributed Track Control System

We have also applied contextual CRA to a distributed track control system [Cheung and Kramer 1994a]. The system was originally proposed by Fisher

	Variation 1		Variation 2		Variation 3	
	#states	time (sec)	#states	time (sec)	#states	time (sec)
conventional CRA	4,083	985.9	103,374	> 3,600	5,743	602.1
contextual CRA	102	6.9	96	8.2	90	3.8

Fig. 27. Performance comparison of conventional and contextual CRA for distributed track control system.

et al. [1992]. It emulates decentralized control of two trains running on a track, which is formed by three rail segments connected in a ring. Each train can only communicate with the rail segment on which it is running. This rail segment communicates with its neighboring segments to ensure that the train can enter safely into the next segment. There are never two trains on the same segment. We have examined three variations of the system, each of which reflects a different compositional hierarchy.

Figure 27 gives the sizes of the largest subsystems constructed and total computational time taken by conventional and contextual CRA techniques on a Sun Sparc/10 workstation. The results show that the performance of conventional CRA is sensitive to different compositional hierarchies. This sensitivity can be dampened by including context constraints in contextual CRA. Interested readers may refer to Cheung and Kramer [1994a] for details. Although our case studies are small in comparison with actual industrial systems, they suggest that contextual CRA is a promising automated technique for compositional reachability analysis.

10. LIMITATIONS OF ALGORITHMICALLY DERIVED INTERFACES

In interface construction, only actions common to the target process are kept in the reduction step (i.e., step (1)) of the algorithm. However, there are cases where actions which not shared with the target still play an important role in constraining the target's behavior. Therefore, the interface constructed may, in some cases, not capture sufficient context constraints to remove a substantial part of the forbidden behavior of a process.

Consider a simple system (Figure 28) with three processes A, B, and C and use of the algorithm to derive an interface for contextual analysis of C. The interface constructed is simply a parallel composition of two dummy processes: I_A and I_B (Figure 29). These processes do not impose any constraints on the behavior of C.

Although action b is not shared by process C, it actually plays an important role in enforcing the alternating execution of actions a and c. In this example, contextual analysis offers the same results as conventional analysis where subsystems are considered to be standalone.

Note that, since the reduced LTSs are processes with a single state, the interface can be constructed with little computational cost. In general, the algorithm requires more computational resources for stricter interfaces.

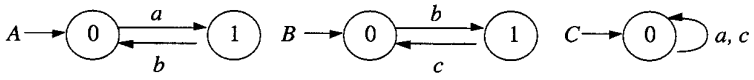

Fig. 28. A simple distributed system.

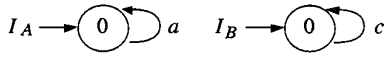

Fig. 29. Reduced LTSs constructed by the algorithm for analysis of C.

Note also that composition of processes A and B prior to deriving the interface process for C would indeed have captured the context constraint.

11. INTERFACE PROCESSES SPECIFIED BY USERS

As discussed above, a limitation of the interface processes that are derived algorithmically is that they may be not be adequate for early removal of forbidden subsystem behavior. Contextual CRA should, therefore, also permit users to utilize their knowledge and intuition of the system behavior. This information can be expressed directly as user-specified interface processes, which can be composed with those derived algorithmically. However, the interface processes specified by users could be incorrect. Their inclusion could result in an incorrect global LTS which does not faithfully represent the behavior of the system. It is, therefore, essential to incorporate a mechanism in the contextual analysis technique to detect erroneous interface processes and to provide an indication of how to correct them.

11.1 Error Detection Mechanism for User-Specified Interface Processes

Given an interface process I and a target system P, the error detection mechanism assumes that

—I introduces no new actions to P, i.e., $\alpha I \subseteq \alpha P$; and
—I is a deterministic finite-state machine free from internal actions and undefined states.

These two assumptions reflect the first and third criteria in the interface theorem (Theorem 6.4.1), including the assumption that users do not specify any undefined states in I. These assumptions can easily be checked and ensured by users. Therefore, in order to ensure that $P \sim P \| I$, the error detection mechanism need only detect the violation of the second criterion, relating to traces. To this end, the interface I is first refined into process I', referred to as the *image interface process* of I. The undefined state π and transitions to it are added automatically using the procedure stated below in Definition 11.1.1. The image process I' can then be used in place of I in contextual CRA of P. Errors will be flagged if the inclusion alters the behavior of P, i.e., $P \| I' \not\sim P$.

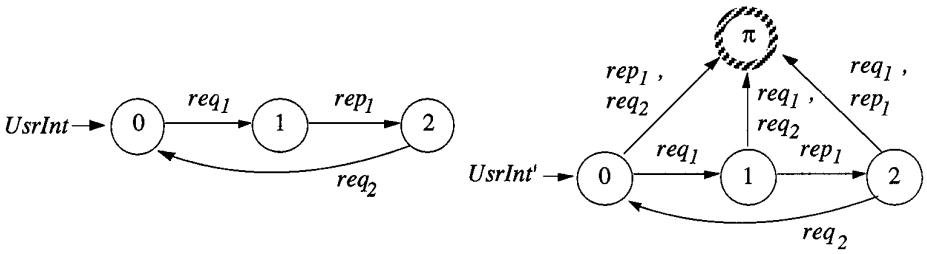

Fig. 30. A user-specified interface processes (left) and its image (right).

Definition 11.1.1. Let $I = \langle S, A, \Delta, i \rangle$ be a totally defined process. We call $I' = \langle S \cup \{\pi\}, A, \Delta', i \rangle$ an image process of I, where

$s \in S - \{\pi\}$, $a \in A$,
if $\Delta(s, a) \neq \varnothing$ then $\Delta'(s, a) = \Delta(s, a)$; otherwise $\Delta'(s, a) = \{\pi\}$.

Figure 30 shows an example of an interface process *UsrInt* and its image *UsrInt'*. Essentially, the image *UsrInt'* keeps all traces in tr(*UsrInt*) and maps those traces not found in tr(*UsrInt*) to undefined ones. This allows all context constraints captured by *UsrInt* to be preserved in *UsrInt'*. The refinement is achieved by inserting an undefined state and six undefined transitions in *UsrInt'*. These additional states and transitions disappear in the analysis if and only if the inclusion of *UsrInt'* in contextual CRA does not alter the overall behavior of *CltSvr*. The correctness of the technique is given by Lemma 11.1.2 and Theorem 11.1.3. Their proofs can be found in Appendix B.

LEMMA 11.1.2. *Let P and I be two totally defined processes where $\alpha I \subseteq \alpha P$, and let I' be an image process of I. Then $P \| I'$ is totally defined if and only if $tr(P \uparrow \alpha I) \subseteq tr(I)$.*

Lemma 11.1.2 states that if an interface process I is overly strict with respect to a given system P, inclusion of its image I' in the contextual CRA would result in a global LTS containing reachable undefined states. This property is utilized in Theorem 11.1.3 to identify problematic interfaces. The theorem ensures that $P \sim P \| I'$ if and only if $P \| I'$ is free from reachable undefined states.

THEOREM 11.1.3. *Let P be a totally defined process and I' be an image process of I that satisfies* (1) $\alpha I \subseteq \alpha P$ *and* (2) *I is a totally defined, deterministic process free of internal action τ. Then $P \sim P \| I'$ if and only if $P \| I'$ is totally defined.*

Figure 31 shows the global LTS constructed if *UsrInt'* is included as the context constraint of $Client_1$. It has a reachable undefined state and hence does not truly represent the behavior of the clients/server system. The example shows that errors can still occur even though the set of traces of the incorrect global LTS (Figure 31) is a superset of that of the correct global LTS (Figure 15).

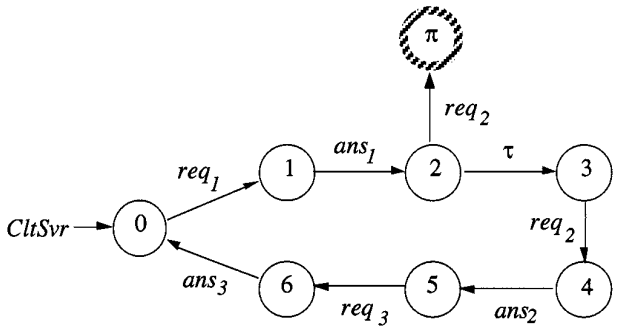

Fig. 31. The global LTS constructed with the inclusion of *UsrInt'*.

The error detection mechanism enumerates at most $|Actions|\times|States|$ undefined transitions during the analysis, where *Actions* and *States* represent the total number of actions in the user-specified interface processes and the total number of states enumerated in the contextual CRA, respectively. The complexity of the mechanism is thus given by $Q(|Actions|\times|States|)$. When users wish to specify interface processes manually, they generally have particular knowledge about the system behavior. Thus we would expect that user-specified interface processes are normally correct or nearly correct. As a result, the number of undefined transitions to be enumerated in the error detection mechanism is generally much less than $|Actions|\times|States|$.

11.2 Location of Error Sources

In general, users may specify a number of interface processes at various stages of contextual CRA to express their knowledge of system behavior related to different subsystems. When the global LTS thus constructed is not totally defined, some legal traces of the original system have been eliminated by the given interface processes. However in many situations users may not be able to identify exactly which transitions have been missing in their interface processes. For the error detection mechanism to be useful, this information should be provided automatically.

To provide this information, the error detection mechanism can be enhanced to keep track of the relation between the undefined transitions in the global LTS and those explicitly inserted in the image interface processes. Let $f_P \subseteq ((S-\{\pi\}) \times A \times \{\pi\}) \rightarrow 2^{((S-\{\pi\})\times A\times\{\pi\})}$ be a mapping associated with a process P, where S and A denote the universal sets of states and actions, respectively. The value of $f_P\,(s, a, \pi)$ indicates the set of undefined transitions in the image interface processes that contribute to the undefined transition (s, a, π) in P.

The mapping f_P is so evaluated that $f_P\,(s, a, \pi)$ equals the empty set $\varnothing$ if (s, a, π) is an *unreachable transition* in P; otherwise,

(1) when P is an image interface process,

$$f_P(s, a, \pi) = \{(s, a, \pi)_P\}$$

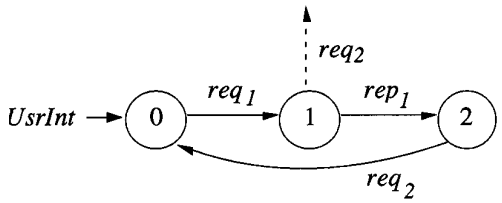

Fig. 32. A missing transition in the interface process *UsrInt*.

(2) when $P = Q \| R$,

$$f_p((s_q, s_r), a, \pi) = f_Q(s_q, a, \pi) \cup f_R(s_r, a, \pi)$$

(3) when $P = Q \uparrow L$,

$$f_P(s, a, \pi) = f_Q(s, a, \pi) \text{ for } a \in L$$

$$f_P(s, \tau, \pi) = f_Q(s, \tau, \pi) \cup \{v | v \in f_Q(s, a, \pi) \text{ for some } a \in \alpha Q - L\}$$

(4) when P is a simplification of Q so that $P \approx Q$,

$$f_P(s, a, \pi) = \{v | v \in f_Q(s', a, \pi)$$

for some s' in Q which identifies a process weakly semantically equivalent to that identified by s}.

The subscript P in a transition $(s, a, \pi)_P$ indicates the process that "owns" the transition. Since the set union operation can be computed using bitwise arithmetics in an imperative programming language, its computational complexity is linear in time and size. Thus the computation of f has the same computational complexity as the enumeration of undefined transitions. It does not increase the computational complexity of the error detection mechanism.

The *CltSvr* in Figure 31 contains only one undefined transition $(2, req_2, \pi)$. Using the evaluation rules above, $f_{CltSvr}(2, req_2, \pi)$ has a value of $\{(1, req_2, \pi)_{UsrInt'}\}$. This indicates that the existence of the undefined transition in *CltSvr* is caused by an omission of a transition labeled with req_2 originating at state 1 of *UsrInt* as shown by a dashed arrow in Figure 32. With this information, users can respecify the interface process accordingly and repeat the contextual CRA.

12. CONCLUSIONS

The analysis of concurrent and distributed systems is a difficult problem. We advocate the use of context constraints in conjunction with compositional reachability analysis. The general idea of applying context constraints in software verification has been suggested previously [Clarke et al. 1989; Graf and Steffen 1990; Krumm 1989; Larsen 1989; Larsen and

Milner 1989]. By focusing on reachability analysis, our contextual CRA technique refines existing work in the two following aspects:

(1) automatic derivation of interface processes which capture context constraints, including proof of its correctness in the framework of compositional reachability analysis; and
(2) detection of incorrect interface processes specified by users including location of the sources of error.

For the first aspect, an interface construction algorithm is presented. The algorithm constructs interface processes from contexts which are generally composed of small or medium-sized primitive processes. An interface theorem is identified to specify the criteria for a transparent interface. For the second aspect, interface processes specified by users are refined into image processes. Undefined states and transitions are inserted automatically in the refinement. These image processes are used instead of the original interfaces specified by users for contextual CRA. Incorrect interface processes results in the existence of undefined transitions in the global LTS. In this case, indication of how to correct the interface processes is provided. Interface processes automatically derived can be combined with those specified by users in the analysis. When free of undefined transitions, the resultant global LTS represents the overall system behavior. Although the complexity of the technique remains exponential in the worst case, preliminary case studies suggest that it provides promising automatable support for the analysis of distributed systems. Results from three case studies were presented.

A limitation of the interface construction algorithm is that it may, in some situations, generate interface processes which are not sufficiently precise to eliminate all forbidden behavioral traces in the composite LTS of a subsystem. Since the effectiveness of contextual CRA depends on the strictness of the interface processes, we are studying different ways to improve the algorithm. One possibility is the use of data flow analysis [Cheung and Kramer 1994b]. Nevertheless, the algorithm presented is simple, easy to implement, and able to generate a correct interface without constructing a composite LTS for the processes in the context. The problem can be partially alleviated by enabling users to exploit their application knowledge and specify their own interface processes.

Contextual CRA can be considered as a refinement of conventional CRA. The former is reduced to the latter if the interface process in analysis always consists of a single state, an empty alphabet, and no transition. Hence, a hybrid analysis technique may be adopted if the software developers of a distributed system wish to locally analyze some subsystems with their contexts and others without their contexts.

We are also interested in applying contextual CRA to models such as Statecharts [Harel 1988], a design formalism using similar communication semantics and graphical notations. In order to further understand the limitations and potential of the technique, we are hoping to apply it to more

complex examples, implementing support tools on workstations, and incorporating it in design environments such as the SAA (System Architect's Assistant), a graphical software development environment for distributed systems [Kramer et al. 1993].

APPENDIX

A. DEFINITIONS IN THE COMPUTATIONAL MODEL

Definition A.1. Given an LTS $\langle S, A, \Delta, p\rangle$ and a state $s \in S$, $\Delta(s, a)$ is defined to be $\{s' | (s, a, s') \in \Delta\}$ if a belongs to A; otherwise $\Delta(s, a) = \{s\}$.

Definition A.2. An LTS $\langle S, A \cup \{\tau\}, \Delta, p\rangle$ is said to be *deterministic* if and only if

$$s, s' \text{ and } s'' \in S,$$
$$(s, a, s') \in \Delta \wedge (s, a\ s'') \in \Delta \Rightarrow s' = s'';$$

otherwise it is said to be *nondeterministic*.

Definition A.3. Let $P = \langle S, A, \Delta, p\rangle$ and $P' = \langle S', A', \Delta', p'\rangle$. $P' = P \uparrow L$ (P restricted to L) is defined as

(1) $S' = S$;
(2) if $A = L$ then $A' = A$; otherwise $A' = A \cap L$;
(3) $s \in S$
$\Delta'(s, a) = \Delta(s, a)$ for $a \in A'$,
$\Delta'(s, \tau) = \Delta(s, \tau) \cup \{s' | s' \in \Delta(s, a)$ for some $a \in A - L\}$;
(4) $p' = p$.

Definition A.4. Let $P = \langle S, A, \Delta, p\rangle$, $P' = \langle S', A', \Delta', p'\rangle$, and $P'' = \langle S'', A'', \Delta'', p''\rangle$. $P'' = P \| P'$ (the composition of P and P') is defined as

(1) $S'' = S \otimes S'$;
(2) $A'' = A \cup A'$;
(3) $(s, s') \in S''$
$\Delta''((s, s'), a) = \Delta(s, a) \otimes \Delta'(s', a)$
$\Delta''((s, s'), \tau) = (\Delta(s, \tau) \otimes \{s'\}) \cup (\{s\} \otimes \Delta'(s', \tau))$;
(4) if $(p = \pi \vee p' = \pi)$ then $p'' = \pi$; otherwise $p'' = (p, p')$.

Here, $\otimes$ denotes a binary *set composition operator* such that for any two sets S and S'

(1) $(S - \{\pi\}) \otimes (S' - \{\pi\}) = (S - \{\pi\}) \times (S' - \{\pi\})$ and
(2) $S \otimes \{\pi\} = \{\pi\} \otimes S = \{\pi\}$.

Formula A.5. The set of traces exhibited by a composition $P_1 \| P_2$ can be evaluated by the following formula given by Hoare [1985]:

$$tr(P_1 \| P_2) = \{t | (t \uparrow \alpha P_1) \in tr(P_1) \wedge (t \uparrow \alpha P_2) \in tr(P_2) \wedge t \in (\alpha P_1 \cup \alpha P_2)^*\},$$

where the set $(\alpha P_1 \cup \alpha P_2)^*$ denotes the set of all finite traces formed from actions in the set $(\alpha P_1 \cup \alpha P_2)$.

Let A be the basic set of actions including τ. Let S be the universal set of states.

Definition A.6. A strong semantic equivalence ($\sim$) is the union of all relations $R \subseteq S \times S$ satisfying that $(P, Q) \in R$ implies

(1) $\alpha P = \alpha Q$;
(2) $P = \Pi$ if and only if $Q = \Pi$; and
(3) for all $a \in A$:
   (a) $P \stackrel{a}{\rightarrow} P'$ implies Q', $Q \stackrel{a}{\Rightarrow} Q'$ and $(P', Q') \in R$;
   (b) $Q \stackrel{a}{\rightarrow} Q'$ implies P', $P \stackrel{a}{\Rightarrow} P'$ and $(P', Q') \in R$.

Let $P \stackrel{a}{\Rightarrow} P'$ denote $P(\stackrel{\tau}{\rightarrow})^* \stackrel{a}{\rightarrow} (\stackrel{\tau}{\rightarrow})^* P'$.

Definition A.7. A weak semantic equivalence ($\approx$) is the union of all relations $R \subseteq S \times S$ satisfying that $(P, Q) \in R$ implies

(1) $\alpha P = \alpha Q$;
(2) $P = \Pi$ if and only if $Q = \Pi$; and
(3) for all $a \in A$:
   (a) $P \stackrel{a}{\Rightarrow} P'$ implies Q', $Q \stackrel{a}{\Rightarrow} Q'$ and $(P', Q') \in R$;
   (b) $Q \stackrel{a}{\Rightarrow} Q'$ implies P', $P \stackrel{a}{\Rightarrow} P'$ and $(P', Q') \in R$.

B. PROOFS OF LEMMAS AND THEOREMS

The following are two properties concerning the trace set of an LTS subject to the restriction operator. Lemma B.1 was suggested by Hoare [1985]. It states that the trace set of a process $(P \uparrow B) \uparrow A$ is the same as that of a process $P \uparrow (B \cap A)$. Lemma B.2 follows directly from Lemma B.1. It states that the trace of a process $(P \uparrow B) \uparrow A$ is the same as that of a process $P \uparrow A$ if set A is a subset of B.

LEMMA B.1. *For any process P and two arbitrary sets of actions A and B,*

$$tr((P \uparrow B) \uparrow A) = tr((P \uparrow (B \cap A)).$$

LEMMA B.2. *For any process P and two arbitrary action sets A and B,*

$$A \subseteq B \Rightarrow tr((P \uparrow B) \uparrow A) = tr(P \uparrow A).$$

PROOF. A direct application of Lemma B.1. □

LEMMA B.3. *For any process P, trace t, and two action sets L and M,*

$$((L \subseteq M) \wedge (t \uparrow M \in tr(P \uparrow M))) \Rightarrow t \uparrow L \in tr(P \uparrow L).$$

PROOF. Let us assume $L \subseteq M$.

$$
\begin{aligned}
& t \uparrow M \in tr(P \uparrow M) \\
\Rightarrow & (t \uparrow M) \uparrow L \in tr((P \uparrow M) \uparrow L) \\
\Rightarrow & t \uparrow (M \cap L) \in tr(P \uparrow L) \quad \text{(by Lemma B.2)} \\
\Rightarrow & t \uparrow L \in tr(P \uparrow L) \quad \square
\end{aligned}
$$

The following are four properties concerning the trace set of a composite LTS. Lemma B.4 states that the trace set of a process is not altered by a multiple composition with itself [Hoare 1985]. Lemma B.5 states that a constituent process has a larger set of trace sequences than the enclosing composite (sub)system with respect to its own alphabet. Lemma B.6 states that if two processes have a larger set of trace sequences than a third process with respect to their own alphabets, then this property is also inherited by their composition.

LEMMA B.4. *For any process* P, $tr(P) = tr(P \| P)$.

LEMMA B.5. *For any two processes* P *and* Q,

$$P = Q \| R \Rightarrow tr(P \uparrow \alpha Q) \subseteq tr(Q).$$

PROOF

$$
\begin{aligned}
& tr(P \uparrow \alpha Q) \\
= & tr((Q \| R) \uparrow \alpha Q) \\
= & \{t \mid (t \uparrow \alpha Q) \in tr(Q) \wedge (t \uparrow \alpha R) \in tr(R) \wedge t \in (\alpha Q)^*\} \\
\subseteq & \{t \mid (t \uparrow \alpha Q) \in tr(Q) \wedge t \in (\alpha Q)^*\} \\
= & tr(Q) \quad \square
\end{aligned}
$$

LEMMA B.6. *For any three processes* P, Q, and R,

$$(tr(P \uparrow \alpha Q) \subseteq tr(Q) \wedge tr(P \uparrow \alpha R) \subseteq tr(R)) \Rightarrow tr(P \uparrow (\alpha Q \cup \alpha R)) \subseteq tr(Q \| R).$$

PROOF

$$
\begin{aligned}
& tr(P \uparrow (\alpha Q \cup \alpha R)) \\
= \; & \{t \mid (t \uparrow (\alpha Q \cup \alpha R)) \in tr(P \uparrow (\alpha Q \cup \alpha R)) \wedge t \in (\alpha Q \cup \alpha R)^*\} \\
\subseteq \; & \{t \mid (t \uparrow \alpha Q) \in tr(P \uparrow \alpha Q) \wedge (t \uparrow \alpha R) \in tr(P \uparrow \alpha R) \wedge t \in (\alpha Q \cup \alpha R)^*\} \\
& \qquad \text{(by Lemma B.3)} \\
\subseteq \; & \{t \mid (t \uparrow \alpha Q) \in tr(Q) \wedge (t \uparrow \alpha R) \in tr(R) \wedge t \in (\alpha Q \cup \alpha R)^* \\
& \qquad \text{(because} \quad tr(P \uparrow \alpha Q) \subseteq tr(Q) \quad \text{and } tr(P \uparrow \alpha R) \subseteq tr(R)) \\
= \; & tr(Q \| R) \quad \square
\end{aligned}
$$

Lemma B.7 states the sufficient condition for the set of traces of a process P to be the same as that of another process formed by $P\|I$.

LEMMA B.7. *For any two processes P and I,*

$$(\alpha I \subseteq \alpha P \wedge tr(P \uparrow \alpha I) \subseteq tr(I)) \Rightarrow tr(P\|I) = tr(P).$$

PROOF

(1) By Lemma B.6, $tr(P) \subseteq tr(P\|I)$.
(2) $tr(P\|I)$

$$\begin{aligned} &= \{t \mid (t \uparrow \alpha P) \in tr(P) \wedge (t \uparrow \alpha I) \in tr(I) \wedge t \in (\alpha P \cup \alpha I)^*\} \\ &\subseteq \{t \mid (t \uparrow \alpha P) \in tr(P) \wedge t \in (\alpha P)^*\} \\ &= tr(P) \quad \square \end{aligned}$$

THEOREM 6.4.1 (INTERFACE THEOREM). *Suppose P and I are two totally defined finite-state processes; and ~ denotes the strong semantic equivalence relation. Then P ~ (P‖I) if*

(1) $\alpha I \subseteq \alpha P$;
(2) $tr(P \uparrow \alpha I) \subseteq tr$(I); and
(3) I is a deterministic process free of internal action τ.

PROOF. To prove the theorem, let us define a binary relation $\mathcal{R}$ over processes by $(P, P\|I) \in \mathcal{R}$ iff conditions (1)–(3) are true.

In the following, we show that $\mathcal{R}$ is a legitimate bisimulation relation R as defined in the strong semantic equivalence in Definition A.6. Let us first assume $(P, P\|I) \in \mathcal{R}$.

PROOF OF CONDITION (1). Since $\alpha I \subseteq \alpha P$, P and $P\|I$ have the same alphabet. Thus condition (1) of the strong semantic equivalence is satisfied.

PROOF OF CONDITION (2). Since both P and I are totally defined, $P\|I$ is also totally defined. The condition (2) of the strong semantic equivalence is thus satisfied.

PROOF OF CONDITION (3). By Lemma B.7, we can show that $\alpha I \subseteq \alpha P$ and $tr(P \uparrow \alpha I) \subseteq tr(I)$ imply that the set of traces of P is the same as that of $P\|I$. Hence $tr(P) = tr(P\|I)$. Thus any action performed by P can also be performed by $P\|I$ and vice versa. To show the satisfaction of condition (3a) of the definition of strong semantic equivalence, let us consider two cases:

Case (1): $P \stackrel{a}{\Rightarrow} P'$ and $a \notin \alpha I$. Since P can transit to P' by executing action a, and a does not belong to the alphabet of I, process $P\|I$ can also transit to process $P'\|I$ by executing a. That is, $P\|I \stackrel{a}{\Rightarrow} P'\|I$.

Since the alphabet of a process is not affected by transition executions, αP equals $\alpha P'$. Thus, $\alpha I \subseteq \alpha P'$. Since action a does not belong to the alphabet of I, set $tr(P \uparrow \alpha I)$ equals $tr(P' \uparrow \alpha I)$. Thus, $tr(P' \uparrow \alpha I) \subseteq tr(I)$. By the

supposition, I is a deterministic process free of internal action. Hence, $(P', P'\|I) \in \mathcal{R}$.

Case (2): $P \xrightarrow{a} P'$ and $a \in \alpha I$. Since P can transit to P' by executing action a, and a belongs to the alphabet of I, P executes action a synchronously with I in the composite process $P\|I$. Since $tr(P)$ equals $tr(P\|I)$, action a can also be executed by $P\|I$. This means that a can be executed by I. If we let I transit into I' after executing action a (i.e., $I \xrightarrow{a} I'$), process $P\|I$ transits into $P'\|I'$ after executing a (i.e., $P\|I \xrightarrow{a} P'\|I'$).

Since an alphabet is not affected by transition executions, αP equals $\alpha P'$. Thus, $\alpha I \subseteq \alpha P'$. Like process I, I' is also a deterministic finite-state process free of internal action. By the determinancy of I, we know that I can only transit into I' with the transition $\xrightarrow{a}$. Therefore, $tr(P' \uparrow \alpha I)$ must be a subset of $tr(I')$; otherwise the condition $tr(P \uparrow \alpha I) \subseteq tr(I)$ is false. Hence, $(P', P'\|I') \in \mathcal{R}$.

Combining cases (1) and (2), we have shown that $\mathcal{R}$ satisfies condition (3a) of the definition of strong semantic equivalence. That is, whenever $(P, P\|I) \in \mathcal{R}$, if $P \xrightarrow{a} P'$ then for some Q' where $Q' = P'\|I$ $(P'\|I')$ when $a \notin \alpha I$ $(a \in \alpha I)$, $P\|I \xrightarrow{a} Q'$ and $(P', Q') \in \mathcal{R}$.

Similarly, we can show that $\mathcal{R}$ satisfies condition (3b) in the definition. Hence, $\mathcal{R}$ is a legitimate bisimulation relation R in the strong semantic equivalence. Hence, $(P, Q) \in \mathcal{R}$ implies $P \sim Q$. □

LEMMA 8.1. *Let $P = U\|V\|R$, and let U' and V' be two processes describing the contextual behavior of U and V, respectively when they separately interact with R. Then $P \approx U'\|V'\|R$.*

PROOF. Let us further assume that I_u and I_v are two interface processes constructed for contextual CRA of U and V such that

(1) $U' \approx U\|I_u$ where I_u is an interface such that $(V\|R) \sim (V\|R)\|I_u$; and
(2) $V' \approx V\|I_v$ where I_v is an interface such that $(U\|R) \sim (U\|R)\|I_v$.

When U' and V' are used to substitute for U and V respectively in the subsequent analysis, the substituted system behaves as $U'\|V'\|R$. However, we know

$$\begin{aligned} & U'\|V'\|R \\ \approx\ & (U\|I_u)\|(V\|I_v)\|R \\ =\ & (U\|I_v)\|(V\|R\|I_u) \\ \sim\ & (U\|I_v)\|(V\|R) \\ =\ & (U\|R\|I_v)\|V \\ \sim\ & U\|V\|R. \end{aligned}$$

Therefore, the substituted system $(U'\|V'\|R)$ and the original system $(U\|V\|R)$ are in the weak semantic equivalence. □

LEMMA 11.1.2. *Let P and I be two totally defined processes where $\alpha I \subseteq \alpha P$, and let I' be an image process of I. Then $P\|I'$ is totally defined if and only if $tr(P \uparrow \alpha I) \subseteq tr(I)$.*

PROOF

Case (1): *if* part. Assume $tr(P \uparrow \alpha I) \subseteq tr(I)$. $tr((P\|I) \uparrow \alpha I')$ equals $\{t | t \in tr(P \uparrow \alpha I') \cap tr(I')\}$ which in turn equals $\{t | t \in tr(P \uparrow \alpha I) \cap tr(I)\} \cup \{t | t \in tr(P \uparrow \alpha I) \cap (tr(I') - tr(I))\}$. The image process I' is so constructed that only those traces in $tr(I') - tr(I)$ can lead I' in an undefined state. Hence, $\{t | t \in tr(P \uparrow \alpha I) \cap tr(I)\}$ and $\{t | t \in tr(P \uparrow \alpha I) \cap (tr(I') - tr(I))\}$ represent the set of defined and undefined traces respectively in $(P\|I') \uparrow \alpha I'$, as $tr(P \uparrow \alpha I) \subseteq tr(I)$, $tr(P \uparrow \alpha I') \cap (tr(I') - tr(I))$ is an empty set. As a result, there are no undefined traces in $(P\|I') \uparrow \alpha I'$. $(P\|I') \uparrow \alpha I'$ is totally defined. Since the restriction operator does not affect the total definedness property of a process, $P\|I'$ is also totally defined.

Case (2): *Only-if* part. Assume $P\|I'$ is totally defined. Let us suppose $tr(P \uparrow \alpha I) \subseteq tr(I)$. This implies that $P \uparrow \alpha I$ can perform a trace t that does not belong to $tr(I)$. We note that any prefix of t can be performed by $P \uparrow \alpha I'$. Let s be a prefix of t such that I can perform all prefixes of s but not the s itself. By the method we construct the image process, s, can also be performed by I'. As a result, s is an undefined trace in I'. Thus s is an undefined trace in $(P \uparrow \alpha \| I')$, which is equal to $(P\|I') \uparrow \alpha I'$. By the properties of the restriction operator, the existence of an undefined trace in $(P\|I') \uparrow \alpha I'$ implies the existence of an undefined trace in $P\|I'$. This contradicts the assumption that $P\|I'$ is totally defined. Thus, the supposition cannot hold. □

LEMMA B.8. *Let P and I be two totally defined processes, and let I' be the image process of I. $P\|I \sim P\|I'$ if $P\|I'$ is totally defined.*

PROOF. Assume $P\|I'$ is totally defined. Let $I = \langle S, A, \Delta, i \rangle$, $I' = \langle S \cup \{\pi\}, A, \Delta', i \rangle$, and $\Delta_u = \Delta' - \Delta$. The image process I' is so constructed that (1) execution of any transitions in Δ_u results in an undefined process and (2) I' and I are in strong semantic equivalence if no transitions in Δ_u can be executed. The assumption that $P\|I'$ is totally defined implies none of the transitions in Δ_u can be executed by the composite process $P\|I'$. Hence $P\|I \sim P\|I'$. □

THEOREM 11.1.3. *Let P be a totally defined process, and let I' be an image process of I that satisfies* (1) *$\alpha I \subseteq \alpha P$ and* (2) *I is a totally defined, deterministic process free of internal action τ. Then $P \sim P\|I'$ if and only if $P\|I'$ is totally defined.*

PROOF. Let us consider two cases: (1) $P\|I'$ is not totally defined and (2) $P\|I'$ is totally defined.

Case (1). Assume $P\|I'$ is not totally defined. It is clear that $P \not\sim P\|I'$ because P is totally defined.

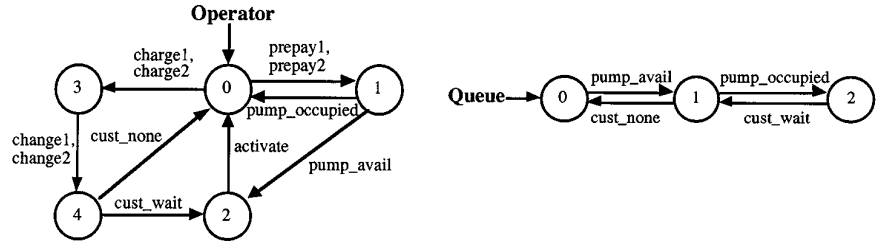

Fig. C.1. Behavior of the *Operator* and *Queue* holding customers' requests.

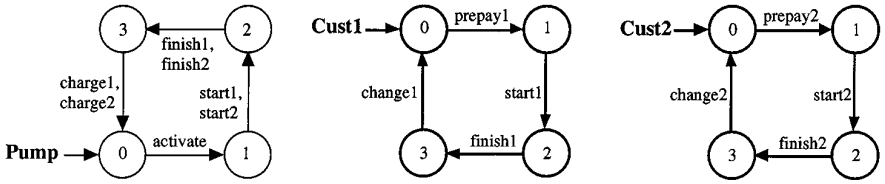

Fig. C.2. Behavior of the *Pump* and the two customers *Cust*1 and *Cust*2.

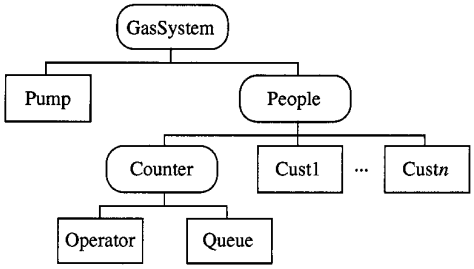

Fig. C.3. Compositional hierarchy of the gas station system.

Case (2). Assume $P \| I'$ is totally defined. By Lemma B.8, $tr(P \uparrow \alpha I) \subseteq tr(I)$. Since P and I satisfy all criteria in the interface theorem, $P \sim P \| I$. By Lemma B.8, $P \sim P \| I$. □

C. GAS STATION EXAMPLE

The gas station example was originally proposed by Helmbold and Luckham [1985]. The example models an automated gas station with an operator, a pump, two customers, and a queue holding customers' requests. Figures C.1 and C.2 give the LTSs presenting the behavior of the operator, request queue,[4] pump, and two customers. The gas station is formed by a composition of the primitive processes *Operator*, *Queue*, *Pump*, *Cust*1, and *Cust*2, using a compositional hierarchy as shown in Figure C.3 [Cheung and Kramer 1995]. In this example, the software developers wish to reason about the global behavior of the system on actions *prepay*1 and *prepay*2. Actions, except *prepay*1 and *prepay*2, internal to each subsystem can thus

[4]If the gas station has more customers, the queue would contain more states in order to accommodate all customers' requests.

be hidden from their external processes. For instance, subsystem *Counter* hides actions *pump_avail*, *pump_occupied*, *cust_none*, and *cust_wait* from processes external to the *Counter*.

ACKNOWLEDGMENTS

We thank the anonymous referees for their careful reviews and constructive comments. Their comments greatly improved both the content and the presentation of the article. We are indebted to Dimitra Giannakopoulou for active discussions and invaluable suggestions.

REFERENCES

AVRUNIN, G. S., BUY, U. A., CORBETT, J. C., DILLON, L. K., AND WILEDEN, J. C. 1991. Experiments with an improved constrained expression toolset. In *Proceedings of the Symposium on Testing, Analysis, and Verification* (*TAV4*). ACM, New York.

BERGSTRA, J. A. AND KLOP, J. W. 1985. Algebra of communicating processes with abstraction. *Theor. Comput. Sci. 37*, 77–121.

BROOKES, S., HOARE, C. A. R., AND ROSCOE, A. 1984. A theory of communicating sequential processes. *ACM 31*, 3, 560–599.

CCITT. 1984. *Integrated Services Digital Network* (*ISDN*) *Recommendation*. CCITT Red Book. Study Group 18, vol. 3, fascicle III.5.

CHEUNG, S. C. AND KRAMER, J. 1994a. An integrated method for effective behaviour analysis of distributed systems. In *Proceedings of the 16th IEEE International Conference on Software Engineering* (*ICSE16*). IEEE, New York.

CHEUNG, S. C. AND KRAMER, J. 1994b. Tractable dataflow analysis for distributed systems. *IEEE Trans. Softw. Eng. 20*, 8 (Aug.).

CHEUNG, S. C. AND KRAMER, J. 1995a. Compositional reachability analysis of finite-state distributed systems with user-specified constraints. In *Proceedings of the 3rd ACM SIGSOFT Symposium on the Foundations of Software Engineering*. ACM, New York.

CHEUNG, S. C. AND KRAMER, J. 1995b. Contextual local analysis in the design of distributed systems. *Int. J. Automat. Softw. Eng. 2*, 1 (Mar.).

CLARKE, E. M., EMERSON, E. A., AND SISTLA, A. P. 1983. Automatic verification of finite state concurrent systems using temporal logic specifications: A practical approach. In *Proceedings of the 10th Annual ACM Symposium on Principles of Programming Languages*. ACM, New York.

CLARKE, E. M., EMERSON, E. A., AND SISTLA, A. P. 1986. Automatic verification of finite state concurrent systems using temporal logic specifications. *ACM Trans. Program. Lang. Syst. 8*, 2, 244–263.

CLARKE, E. M., LONG, D. E., AND MCMILLAN, K. L. 1989. Compositional model checking. In *Proceedings of the 4th Annual Symposium on Logic in Computer Science*.

DENICOLA, R. AND HENNESSY, M. 1984. Testing equivalences for processes. *Theor. Comput. Sci. 34*, 83–133.

FISCHER, S., SCHOLZ, A., AND TAUBNER, D. 1992. Verification in process algebra of the distributed control of track vehicles—A case study. In *Proceedings of the 4th International Workshop on Computer-Aided Verification* (*CAV'92*). ACM, New York.

GARMAN, J. R. 1981. The bug heard around the world. *ACM SIGSOFT Softw. Eng. Notes 6*, 5.

GHEZZI, C., JAZAYERI, M., AND MANDRIOLI, D. 1991. *Fundamentals of Software Engineering*. Prentice-Hall, Englewood Cliffs, N.J., chapter 6.

GRAF, S. AND STEFFEN, B. 1990. Compositional minimization of finite state systems. In *Proceedings of the 2nd International Conference of Computer-Aided Verification*. ACM, New York.

HAREL, D. 1988. On visual formalisms. *Commun. ACM 31*, 5, 514–530.

HELMBOLD, D. AND LUCKHAM, D. 1985. Debugging Ada tasking programs. *IEEE Softw. 2*, 2 (Mar.), 47–57.

HENNESSY, M. 1988. *Algebraic Theory of Processes*. MIT Press, Cambridge, Mass.

HOARE, C. A. R. 1985. *Communicating Sequential Processes*. Prentice-Hall, Englewood Cliffs, N.J.

HOLZMANN, G. 1991. *Design and Validation of Computer Protocols*. Prentice-Hall, Englewood Cliffs, N.J., chapters 8 and 11.

HOPCROFT, J. E. AND ULLMAN, J. D. 1979. *Introduction to Automata Theory, Languages, and Computation*. Addison-Wesley, Reading, Mass.

KEMPPAINEN, J., LEVANTO, M., VALMARI, A., AND CLEGG, M. 1992. "ARA" puts advanced reachability analysis techniques together. In *Proceedings of the 5th Nordic Workshop on Programming Enviornment Research*. Also available as Tech. Rep. 14, Software Systems Laboratory, Tampere Univ. of Technology, Tampere, Finland.

KRAMER, J., MAGEE, J., NG, K., AND SLOMAN, M. 1993. The system architect's assistant for design and construction of distributed systems. In *Proceedings of the 4th IEEE Workshop on Future Trends of Distributed Computing Systems*. IEEE, New York.

KRUMM, H. 1989. Projections of the reachability graph and environment models. In *Proceedings of the International Workshops on Automatic Verification Methods for Finite State Systems*. Lecture Notes in Computer Science, vol. 407. Springer-Verlag, Berlin.

LARSEN, K. AND MILNER, R. 1987. Verifying a protocol using relativized bisimulation. In *Proceedings of the 14th International Colloquium on Automata, Languages and Programming*. Lecture Notes in Computer Science, vol. 267. Springer-Verlag, Berlin.

LARSEN, K. G. 1989. Compositional theories based on an operational semantics of contexts. In *Proceedings of the REX Workshop on Stepwise Refinement of Distributed Systems: Models, Formalisms, Correctness*. Lecture Notes in Computer Science, vol. 430. Springer-Verlag, Berlin.

MALHOTRA, J., SMOLKA, S. A., GIACALONE, A., AND SHAPIRO, R. 1988. A tool for hierarchical design and simulation of concurrent systems. In *Proceedings of the BCS-FACS Workshop on Specification and Verification of Concurrent Systems*. British Computer Society, Swindon, England.

MILNER, R. 1989. *Communication and Concurrency*. P:rentice-Hall, Englewood Cliffs, N.J.

MILNER, R., PARROW, J., AND WALKER, D. 1989. A calculus of mobile processes part I and II. Tech. Rep. Univ. of Edinburgh, Edinburgh, U.K. June.

PETERSON, J. L. 1981. *Petri Net Theory and the Modelling of Systems*. Prentice-Hall, Englewood Cliffs, N.J.

RABINOVICH, A. 1992. Checking equivalences between concurrent systems of finite agents. In *Proceedings of the 19th International Colloquium on Automata, Languages and Programming*. Lecture Notes in Computer Science, vol. 623. Springer-Verlag, Berlin.

SABNANI, K. K., LAPONE, A. M., AND UYAR, M. Ü. 1989. An algorithmic procedure for checking safety properties of protocols. *IEEE Trans. Commun. 37*, 9 (Sept.), 940–948.

SMOLKA, S. A. 1984. Analysis of communication finite state processes. Ph.D. thesis, CS-84-05, Dept. of Computer Science, Brown Univ., Providence, R.I.

TAI, K. C. AND KOPPOL, P. V. 1993. An incremental approach to reachability analysis of distributed programs. In *Proceedings of the 7th IEEE International Workshop on Software Specification and Design*. IEEE, New York.

TAYLOR, R. N. 1983a. Complexity of analyzing the synchronization structure of concurrent programs. *Acta Informatica 19*, 57–84.

TAYLOR, R. N. 1983b. A general-purpose algorithm for analyzing concurrent programs. *Commun. ACM 26*, 362–376.

VALMARI, A. 1991. Compositional state space generation. Tech. Rep., A-1991-5, Dept. of Computer Science, Univ. of Helsinki, Finland. Oct.

VALMARI, A. 1992. Alleviating state explosion during verification of behavioural equivalence. Tech. Rep., A-1992, Dept. of Computer Science, Univ. of Helsinki, Finland. Aug.

YEH, W. J. 1993. Controlling state explosion in reachability analysis. Tech. Rep., SERC-TR-147-P, SERC, Purdue Univ., West Lafayette, Ind. Dec.

YEH, W. J. AND YOUNG, M. 1991. Compositional reachability analysis using process algebra. In *Proceedings of the Symposium on Testing, Analysis, and Verification* (*TAV4*). ACM, New York.

Received February 1994; revised July 1994, June 1995, and February 1996; accepted May 1996